DEPSTAR

Devang Patel Institute of Advance Technology and Research

(A Constitute Institute of CHARUSAT)

Certificate

This is to certify that

Mr./Mis. Vansani Krish Alpeshbhai

of DCSE - 2 *Class,*

ID. No. 25DCS140 *has satisfactorily completed*

his/ her term work in CEUE 203 - OOPS *for*

the ending in Dec 2026 /2027

Date : 22 - 10 - 26

Sign. of Faculty

Head of Department

Devang Patel Institute of Advance Technology and Research

DEPSTAR (A Constitute Institute of CHARUSAT)

Name: Vangani Krish ID. No. 25DCS140

INDEX

No.	DATE	SUBJECT	PAGE	MARKS	SIGNATURE
1	17/7/26	1. Java Basics	2-10		
2	17/7/26	2. Classes & objects	11-24		
23	7/8/26	3. String	25-35		
4	14/8/26	4. Inheritance	36-52		
5	21/8/26	5. Annotation	53-61		
6	18/9/26	6. Interface	62-72		
7	25/9/26	7. Exception han.	73-84		
8	25/9/26	8. multithreading	85-97		
9	25/9/26	9. thread	98-107		
10	9/10/26	10. file handling	108-120		
11	9/10/26	11. file operating	121-135		
12	16/10/26	12. collection	136-144		
13	16/10/26	13. Generics	145-154		
14	16/10/26	14. Integration	155-188		

CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CEUE203 Object Oriented Programming

DEPSTAR

PRACTICAL – 1

Problem Statement:

1.Vending machine (VendingMachine.java, main in this file) (a) Define an enum Coin with constants ONE, TWO, FIVE, TEN. (b) In main, set a snack price of 15 and a running total of 0; create a Scanner. (c) Loop: read a coin name, use a switch expression to convert the Coin to its value (ONE→1, TWO→2, FIVE→5, TEN→10), add it to the total, and print the total so far. (d) Stop the loop once the total reaches 15 or more. (e) Print the change to return (total − 15). Expected output: TEN then FIVE prints “Paid. Change: 0”; TEN then TEN prints “Paid. Change: 5”.

Code:

```
import java.util.Scanner;

enum Coin{

ONE,TWO,FIVE,TEN

}

class prac1_1{

public static void main(String[] args) {

Scanner sc=new Scanner(System.in);

int price=15;

int total=0;


   while(total<price){

      Coin coin=Coin.valueOf(sc.next().toUpperCase());

      int value=switch(coin){

         case ONE->1;

         case TWO->2;

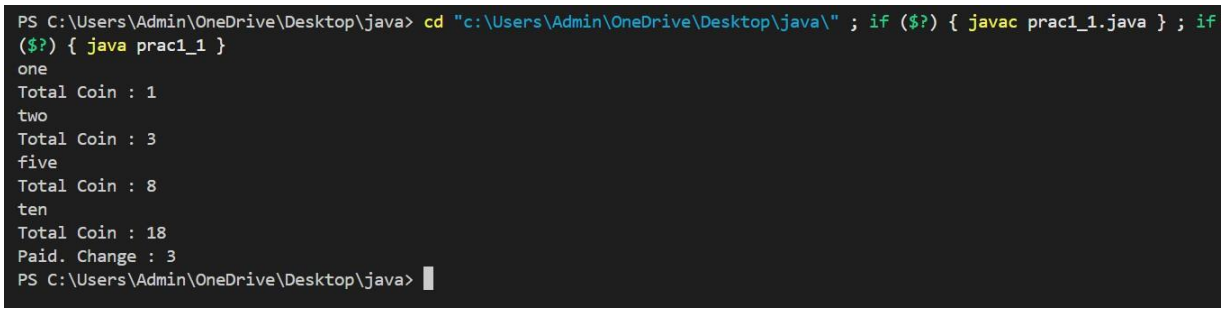
         case FIVE->5;

         case TEN->10;
```

```
        };
        total+=value;
        System.out.println("Total Coin : "+ total);

    }
    System.out.println("Paid. Change : "+(total-price));
}
}
```

Output/SS:



Practical 1.2:

Problem Statement:

2.Toll booth (TollBooth.java, main in this file) (a) Define a record Vehicle(String number, String type). (b) In main, keep a running toll total and three counters (bike, car, truck). (c) Loop until the user types “done” for the number: build a Vehicle, use a switch expression on its type for the toll (bike→20, car→50, truck→150), add to the total, and increment that type’s counter. (d) After the loop, print the total toll and the type with the highest count. Expected output: two cars + one bike → “Total toll: 120”, “Most frequent: car”.

Code:

```
import java.util.Scanner;

record Vehical(String number,String type){}

class prac1_2{
```

```
public static void main(String[] args) {
        int tollTotal=0;
        int bike=0;
        int car=0;
        int truck=0;
        int i=0;
        Scanner sc=new Scanner(System.in);
        Vehical []v=new Vehical[100];
        while(true){
            System.out.println("Enter Vehical Number And Type And Enter
Done to Exists");
            String number=sc.next();

            if(number.equalsIgnoreCase("done"))
            break;

             String type=sc.next();


                v[i]=new Vehical(number,type);
          int toll=switch(v[i].type()){
          case "bike"->{
           bike++;
           yield 20;

           }
           case "car"->{
              car++;
```

```
            yield 50;

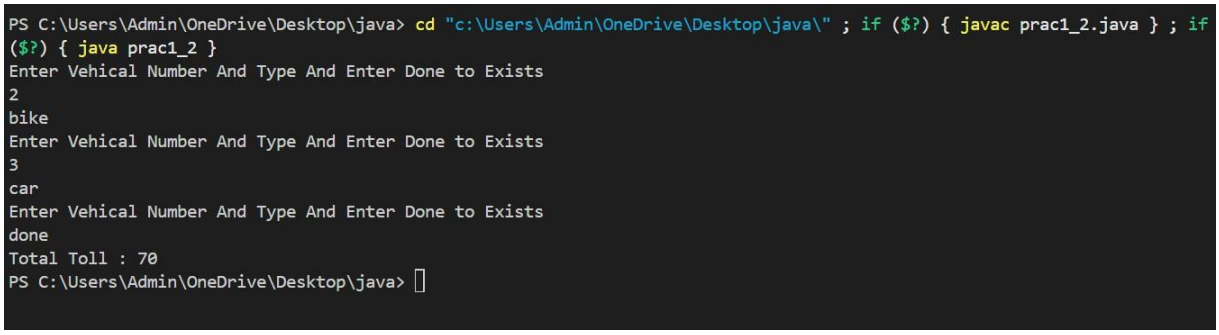
        }
        case "truck"->{
            truck++;
            yield 150;

        }
        default->0;

};
         tollTotal+=toll;
i++;
      }
      System.out.println("Total Toll : "+ tollTotal);
      if(bike>car&&bike>truck)
      System.out.println("Most frequent : Bike");
      if(bike<car&&car>truck)
      System.out.println("Most frequent : Car");
      if(truck>car&&bike<truck)
      System.out.println("Most frequent : Truck");

}
}
```

Output/SS:



MINIBANK-1

Problem Statement:

MiniBank is the console banking application you will build across the whole semester. In
this first lab you set up your tools and create the project and its menu shell — the screen the
user navigates. Every later lab attaches one real banking feature behind these menu options.

1.Set up your environment: open the project in GitHub Codespaces (or install JDK 17+ and
an IDE locally), and create a private GitHub repository named oop-minibank-<rollno>;
add the faculty as a collaborator.

2.Create a public class named MiniBank that contains the main method.

3.Define a record named BankInfo with two String fields, name and branch (a record is a
short, read-only class; Java auto-generates its constructor and field accessors). Print one
BankInfo object as the application header.

4.Define an enum named MenuOption with the constants OPEN_ACCOUNT, DEPOSIT,
WITHDRAW, TRANSFER and EXIT — these are the actions MiniBank will offer.

5.Display the numbered menu to the user and read the chosen number with a Scanner.

6.Use a switch expression on the choice to print a short placeholder for that option (for
example, “Deposit — to be implemented in a later lab”). Keep showing the menu in a
loop until the user selects EXIT, then print a goodbye message.

Code:

```
import java.util.Scanner;
record BankInfo(String Name,String Branch){}
enum MenuOption{
OPEN_ACCOUNT,
DEPOSIT,
WITHDRAW,
TRANSFER,
EXIT
}
class Customer{
    private String fieldName;
    private String email;
    private String mobileNo;
    private final String field customerId;
}
public class minibank{
```

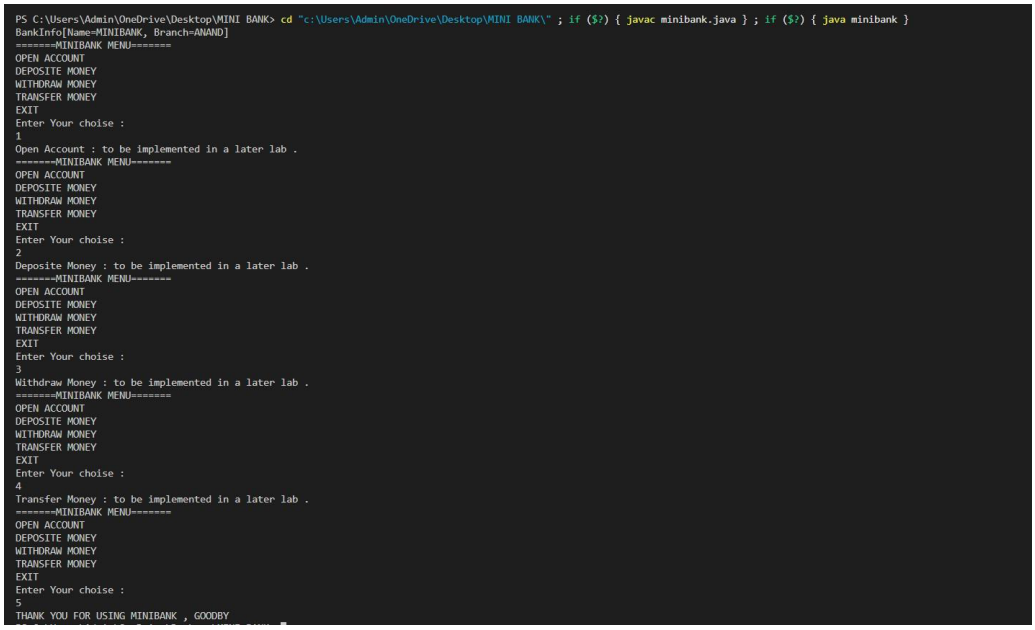
```
public static void main(String []arg){
     Scanner sc=new Scanner(System.in);
    BankInfo bank=new BankInfo("MINIBANK","ANAND");
    System.out.println(bank);
    boolean x=true;
    while(x){
      System.out.println("==MINIBANK MENU==");
       System.out.println("OPEN ACCOUNT");
       System.out.println("DEPOSITE MONEY");
       System.out.println("WITHDRAW MONEY");
       System.out.println("TRANSFER MONEY");
       System.out.println("EXIT");
       System.out.println("Enter Your choise :");
        int a=sc.nextInt();
        String msg=switch(a){
        case 1->{
            yield "Open Account : to be implemented in a later lab . ";
         }
         case 2->{
            yield "Deposite Money : to be implemented in a later lab .";
         }
          case 3->{
            yield "Withdraw Money : to be implemented in a later lab .";
         }
          case 4->{
            yield "Transfer Money : to be implemented in a later lab .";
```

```
            }
            case 5->{
                x=false;
                yield "THANK YOU FOR USING MINIBANK , GOODBY";



            }
            default->{
                yield "INVALID CHOISE , PLEASE ENTER CORRECT
CHOISE";
            }
            };
            System.out.println(msg);
        }
}
}
```

Output/SS:



Key Questions and Solution

1.What is the role of the JVM, and what file does the javac compiler produce?

Answer: The JVM (Java Virtual Machine) is responsible for executing Java bytecode. It makes Java platformindependent because the same bytecode can run on any system that has a compatible JVM. The javac compiler converts a Java source file (.java) into bytecode, which is stored in a .class file.

2.How does a switch expression differ from a traditional switch statement?

Answer: A switch expression can directly return a value, while a traditional switch statement is mainly used to execute a block of code. Switch expressions can use -> instead of case with break, making the code shorter and reducing the possibility of fall-through errors. Example: String result = switch (choice) { case 1 -> "Deposit"; case 2 -> "Withdraw"; default -> "Invalid choice"; };

3.Why is an enum a good choice for a fixed set of menu options, and a record for read-only data?

Answer: An enum is suitable for a fixed set of menu options because it allows us to define a limited and predefined set of constants, such as DEPOSIT, WITHDRAW, and EXIT. It improves type safety and makes the code easier to understand. A record is suitable for read-only data because it is designed to store immutable data. It automatically provides a constructor, accessor methods, equals(), hashCode(), and toString(), reducing the amount of code that needs to be written.

PRACTICAL – 2

Problem Statement:

1.Smart thermostat (Thermostat.java, main in this file) (a) Create class Thermostat with private String location, private int temperature, private static final int MIN=16 and MAX=30, and private static int activeCount=0. (b) Constructor(location, startTemp): store location; set temperature to startTemp if it is within MIN..MAX else 22; increment activeCount. (c) Constructor(location): chain with this(location, 22). (d) raise(): if temperature < MAX add 1, else print “Already at maximum (30)”. (e) lower(): if temperature > MIN subtract 1, else print “Already at minimum (16)”. (f) Add getTemperature() (no setter) and static getActiveCount(). (g) In main: create two thermostats, call raise() 10 times then lower() 20 times in loops (printing the temperature each time), then print getActiveCount(). Expected output: temperature rises to 30 then “Already at maximum”, falls to 16 then “Already at minimum”, count 2.

Code:

```
import java.util.Scanner;
class thermostate{
    private String location;
    private int temp;
    private static final int MIN=16;
    private static final int MAX=30;
    private static int activeCount=0;

    thermostate(String location,int startTemp){
        this.location=location;
```

```
        if(startTemp<=30&&startTemp>=16){
         temp=startTemp;
        }
        else{
         temp=22;
        }
        activeCount++;
    }
    thermostate(String location){
       this(location,22);
    }
    void rais(){
       if(temp<MAX){
          temp++;
       }
       else{
          System.out.println("Already at maximum (30)");
       }
    }
    void lower(){
       if(temp>MIN){
          temp--;
       }
       else{
          System.out.println("Already at minimum (16)");
       }
    }
```

```
        void display() {
        System.out.println(location);
    }
    int getTemprature(){
        return temp;
    }
    static int getActiveCount(){
        return activeCount;
    }
    };
    class prac2_1{
    public static void main(String []arg){
        Scanner sc=new Scanner(System.in);
        System.out.println("Enter Location ");
        String location=sc.nextLine();
        System.out.println("Enter Temprature ");
        int temp=sc.nextInt();
        thermostate t=new thermostate(location,temp);
        thermostate t1=new thermostate(location,temp);
        for(int i=0;i<10;i++){
           t1.rais();
           System.out.println("Temprature : "+t1.getTemprature());
        }
        for(int i=0;i<20;i++){
           t1.lower();
           System.out.println("Temprature : "+t1.getTemprature());
        }
```

```
        System.out.println("Active Count : "+thermostate.getActiveCount());


    }
}
```

Output/SS:



PRACTICAL-2.2

Problem Statement :

2.Cinema show (CinemaShow.java, main in this file) (a) Create class CinemaShow with private String title, private int seatsAvailable, private final int capacity, and private static int totalBooked=0. (b) Constructor(title, capacity): set both and seatsAvailable = capacity. Constructor(title): chain with this(title, 100). (c) book(int n): if n ≤ seatsAvailable, reduce seatsAvailable by n, add n to totalBooked, return true; else return false unchanged. (d) cancel(int n): increase seatsAvailable by n but never above capacity. (e) Add getSeatsAvailable() and static getTotalBooked(). (f) In main: run a sequence of book and cancel calls, printing the result and seats after each, then print getTotalBooked(). Expected output: booking more

than available prints false and leaves seats unchanged; totalBooked equals the successful bookings.

Code:

```
import java.util.Scanner;
class CinemaShow{
private String tital;
private int seatAvailable;
private final int capacity;
private static int totalBook=0;

   CinemaShow(String tital,int capacity){
      this.tital=tital;
      this.capacity=capacity;
      seatAvailable=capacity;
   }
   CinemaShow(String tital){
      this(tital,100);
   }
   boolean book(int n){
         if(n<=seatAvailable){
         seatAvailable-=n;
         totalBook+=n;
          return true;
         }
         else{
```

```
            return false;
           }
    }
    void cancle(int n){
       seatAvailable+=n;
       if(capacity<seatAvailable)
       seatAvailable=capacity;



    }
    int getSeatAvailable(){
       return seatAvailable;
    }
    static int getTotalBook(){
       return totalBook;
    }

};
class prac2_2{
public static void main(String []arg){
       Scanner sc=new Scanner(System.in);
       System.out.println("Enter The Tital Of CinemaShow");
       String tital=sc.nextLine();
        System.out.println("Enter The Capacity Of CinemaShow");
       int capacity=sc.nextInt();
      CinemaShow c= new CinemaShow(tital,capacity);
```

```
        while(true){
            System.out.println("Type For :"+"\n"+"1.Book Ticket
"+"\n"+"2.Cancle Ticket"+"\n"+"3.Get Available Ticket"+"\n"+"4.Total
Booked"+"\n"+"5.Exists");
        int a=sc.nextInt();
              switch(a){
            case 1:
                System.out.println("Enter Howmany Ticket You Want To Book");
                int n=sc.nextInt();
                if(c.book(n)){
                   System.out.println(true);
                    System.out.println("Your Ticket Booked Successfully");
               }
                else{
                    System.out.println(false);
                    System.out.println("Booking Failed");
                }
                 System.out.println("Available Tickets : "+c.getSeatAvailable());
                 break;

            case 2:
                 System.out.println("Enter Howmany Ticket You Want To
Cancle");
                int b=sc.nextInt();
                c.cancle(b);
                    System.out.println("Your Ticket Cancle Successfully");

                 System.out.println("Available Tickets : "+c.getSeatAvailable());
```

```
                break;
            case 3:
                System.out.println("Available Tickets : "+c.getSeatAvailable());
                break;
            case 4:
                System.out.println("Total Booked :
"+CinemaShow.getTotalBook());
                break;
            case 5:
                System.out.println("Exists . ThankYou ");
                return;
            default:
                System.out.println("Invaid Choise");

        }

        }
    }
}
```

Output:

```
Enter The Tital Of CinemaShow
Bahubali
Enter The Capacity Of CinemaShow
100
Type For :
1.Book Ticket
2.Cancle Ticket
3.Get Available Ticket
4.Total Booked
5.Exists
1
Enter Howmany Ticket You Want To Book
5
true
Your Ticket Booked Successfully
Available Tickets : 95
Type For :
1.Book Ticket
2.Cancle Ticket
3.Get Available Ticket
4.Total Booked
5.Exists
2
Enter Howmany Ticket You Want To Cancle
1
Your Ticket Cancle Successfully
Available Tickets : 96
Type For :
1.Book Ticket
2.Cancle Ticket
3.Get Available Ticket
4.Total Booked
5.Exists
3
Available Tickets : 96
Type For :
1.Book Ticket
2.Cancle Ticket
3.Get Available Ticket
4.Total Booked
5.Exists
4
Total Booked : 5
Type For :
1.Book Ticket
2.Cancle Ticket
3.Get Available Ticket
4.Total Booked
5.Exists
5
```

MINIBANK-2

Problem Statement :

In Practical 1 you built the empty menu shell. Now you create the data the bank works on —

the Customer and Account classes. Using encapsulation, an account's balance can change

only through deposit() and withdraw(), never directly. These two classes are the foundation

every later lab extends.

1.Create a class named Customer with private fields name, email and mobile (all String)

and a private final field customerId (String). Use a private static long field customerCounter and a private static method generateCustomerId() to create ids such as

"CUST101". Add a constructor and public getter methods.

2.Create a class named Account with private fields: a final String accountNumber, a String

ownerName, a long balance (whole rupees) and a boolean active. Generate accountNumber with a private static counter (for example “AC0001”). 3.Add two constructors to Account: one taking ownerName and an opening balance; one
taking only ownerName that calls the first using this(ownerName, 0).

4.Add a public method deposit(long amount) that adds amount to balance. Add a public
method withdraw(long amount) that subtracts amount and returns true when the balance
is sufficient, otherwise returns false without changing the balance. Add getter methods.

Do NOT add a public setter for balance.

5.In main, create three Account objects inside an Account[] array, perform a few deposits
and withdrawals, and print each balance.

Code:

```
import java.util.Scanner;
class Customer{
  private String name;
  private String email;
  private String mobileNo;
  private final String customerID;
private static long customerCo=100;

   private static String getCustomerID(){

      return "CUST"+customerCo++;
```

```
    }

    Customer(String name,String email,String mobileNo){
        this.name=name;
        this.email=email;
        this.mobileNo=mobileNo;
        this.customerID=getCustomerID();
    }
    public String getName(){
        return name;
    }
        public String getEmail(){
        return email;
    }
        public String getMobileNo(){
        return mobileNo;
    }

public String CustomerID(){
        return customerID;
}
}
class Account{

    private final String accountNumber;
    private String name;
    private long balance;
```

```
private boolean active;
private static long counter=100;

private String accountNumber(){
   return "AC"+counter++;
}

Account(String name,long balance){
        this.name=name;
        this.balance=balance;
        this.accountNumber=accountNumber();
       this.active=true;
}
Account(){
   this("",0);
}
public void depositeMoney(long bal){
          balance+=bal;
}
public boolean withdrawMoney(long bal){
 if(bal<=balance){
       balance-=bal;
       return true;

   }
   else{
       return false;
```

```
    }
  }
  public String getaccountNumber(){
  return accountNumber;
  }
     public String Name(){
    return name;
  }
  public boolean active(){
       return active;
  }
  public long balance(){
    return balance;
  }
}
class Main{
public static void main(String[] args){
   Scanner sc=new Scanner(System.in);
        Account[] a=new Account[3];

  for(int i=0;i<3;i++){
     System.out.println("Enter Name : ");
        String nm=sc.next();
        System.out.println("Enetr Bank Balance : ");
        int bal=sc.nextInt();
        a[i]=new Account(nm,bal);
  }
```

```
        a[0].depositeMoney(1000);
        a[1].depositeMoney(2000);
        a[2].depositeMoney(3000);

        a[0].withdrawMoney(500);
        a[1].withdrawMoney(1000);
        a[2].withdrawMoney(1500);

    System.out.println(a[0].balance());
System.out.println(a[1].balance());
System.out.println(a[2].balance());

}
}
```

Output :

```
PS C:\Users\Admin\OneDrive\Desktop\MINI BANK> cd "c:\Users\Admin\OneDrive\Desktop\MINI BANK\" ; if ($?) { javac Main.java } ; if ($?) { java Main }
Enter Name :
Krish
Eneter Bank Balance :
2000
Enter Name :
Het
Eneter Bank Balance :
3000
Enter Name :
Ayush
Eneter Bank Balance :
5000
2500
4000
6500
PS C:\Users\Admin\OneDrive\Desktop\MINI BANK>
```

Key Questions and Solution :

1.What is encapsulation, and how do private fields with public getters enforce it?

Answer: Encapsulation is the process of hiding the internal data of a class and controlling access to it. Private fields cannot be accessed directly from outside the class. Public getter methods provide controlled access to read the data. This protects the data and improves security and maintainability.

2.Why is accountNumber declared final?

Answer: accountNumber is declared final because an account number should be unique and should not change after the account is created. Once a value is assigned to a final variable, it cannot be reassigned.

3.What is the difference between a static field and an instance field when generating IDs?

Answer: A static field belongs to the class and is shared by all objects. It is useful for generating unique IDs because all objects can use the same counter. An instance field belongs to an individual object, so every object has its own separate copy. Therefore, it is not suitable for maintaining a common ID counter for all objects. Example: private static int nextId = 1001; Here, nextId is shared by all objects and can be used to generate unique account numbers.

PRACTICAL – 3

Problem Statement :

Distinct points (Point.java + Driver.java with main)

(a)Create class Point with private int x, y and a constructor.

(b)Override toString() to return “(x, y)”.

(c)Override equals(Object) to be true when both x and y match, and hashCode() to agree

(e.g. Objects.hash(x, y)).

(d)In Driver.main: build a Point[] with some repeated coordinates.

(e)Count distinct points: for each point, use equals() to check whether it already appeared earlier; count those that did not.

(f)Print “Distinct: N”.

Code :

```
import java.util.Objects;
class Point{
private int x;
private int y;
Point(int x,int y){
        this.x=x;
        this.y=y;
}
public String toString(){
      return "("+x+","+y+")";
}
public boolean equals(Object obj){
      if(this==obj)
      return true;
      if(!(obj instanceof Point))
      return false;
      Point p=(Point)obj;
      return p.x==this.x&&p.y==this.y;
}
public int hashCode(){
```

```
        return Objects.hash(x,y);
}
}
public class prac3_1{
public static void main(String[] args){
        Point[] p={
                new Point(1,2),
                new Point(2,3),
                new Point(1,3),
                new Point(4,3),
                new Point(1,6),
                new Point(1,3),
                new Point(4,3),
                new Point(1,9),
                 new Point(0,3)};
         int count=0;
        for(int i=0;i<p.length;i++){
           boolean ans=false;

           for(int j=0;j<i;j++){
              if(p[i].equals(p[j])){
                 ans=true;
                 break;
              }

           }
           if(!ans)
```

```
            count++;
        }
        System.out.println("Distinct : "+count);
}
}
```

Output :

```
PS C:\Users\Admin\OneDrive\Desktop\java> cd "c:\Users\Admin\OneDrive\Desktop\java\" ; if ($?) { javac prac3_1.java } ; i
 ($?) { java prac3_1 }
Distinct : 7
PS C:\Users\Admin\OneDrive\Desktop\java>
```

Practical-3.2

Problem Statement :

Duplicate card check (Card.java + Driver.java)

(a)Create class Card with private String rank, suit and a constructor.

(b)Override toString() to return rank + " of " + suit (e.g. "Queen of Hearts").

(c)Override equals()/hashCode() by rank + suit.

(d)In Driver.main: add cards to a Card[] one at a time; before adding each, use equals()
to check it against the earlier cards.

(e)Print "Duplicate found: <card>" the first time a repeat appears.

Code :

```
import java.util.Objects;
class Card{
private String rank;
private String suit;
```

```
    Card(String rank,String suit){
        this.rank=rank;
        this.suit=suit;
    }
    public String toString(){
        return rank + " Of " + suit;
    }
    public int hashCode(){
        return Objects.hash(rank,suit);
    }
        public boolean equals(Object obj) {
        if (this == obj)
            return true;
        if (!(obj instanceof Card))
            return false;
         Card c = (Card) obj;

        return rank.equals(c.rank) && suit.equals(c.suit);
}
}
public class prac3_2{
     public static void main(String[] args){
        int count=0;
        Card[] c={
                new Card("Ace", "Hearts"),
            new Card("King", "Spades"),
            new Card("Ace", "Spades"),
```

```
            new Card("Queen", "Hearts"),
            new Card("Ace", "Spades"),
            new Card("King", "Clubs")
        };
        for(int i=0;i<c.length;i++){
            boolean ans=false;
            for(int j=0;j<i;j++){
            if(c[i].equals(c[j])){
                    ans=true;
                    break;
            }
            }
            if(ans){
            System.out.println("Duplicate Found : "+c[i]);
            }

        }
    }
}
```

Output :

```
PS C:\Users\Admin\OneDrive\Desktop\java> cd "c:\Users\Admin\OneDrive\Desktop\java\" ; if ($?) { javac prac3_2.java } ; if
 ($?) { java prac3_2 }
Duplicate Found : Ace Of Spades
PS C:\Users\Admin\OneDrive\Desktop\java>
```

MINIBANK-3

Problem Statement :

In Practical 2 you created the Account and Customer classes. Here you make their objects behave correctly — they print as readable text (toString) and are compared by account number (equals/hashCode) — and you add a nested Address class. Correct equals/hashCode is essential because Practical 12 will store accounts in collections that rely on it.

1.In Account, override the toString() method to return a readable line containing accountNumber, ownerName and balance.

2.In Account, override equals(Object o) and hashCode() so that two accounts are equal when their accountNumber values are equal.

3.In Customer, add a public static nested class named Address with String fields line, city and pincode plus getters; add an Address field to Customer and a getAddress() method.

4.In Customer, add a clone() method (implement Cloneable) that returns a copy of the customer.

5.In main, print accounts using toString(), compare two Account objects with equals(), and use instanceof to check an object's type.

Code :

```
class Account{
    private final String accountNumber;
    private String name;
    private long balance;

    Account(String accountNumber,String name,long balance){
        this.accountNumber=accountNumber;
        this.name=name;
        this.balance=balance;
    }
    public String toString(){
```

```
    return "Account Number : "+accountNumber +
        " Onwer Name : "+name +
        " Account Balance : "+balance;
}
public boolean equals(Object o){
        if(this==o)
        return true;
        if(!(o instanceof Account))
        return false;
        Account other=(Account)o;
return this.accountNumber.equals(other.accountNumber); }
public int hashCode(){
        return accountNumber.hashCode();
}
}
class Customer implements Cloneable{
private String name;
private Address address;

    Customer(String name,Address address){
        this.name=name;
        this.address=address;
    }

    static class Address{
            private String line;
```

```
        private String city;
        private String pincode;

        Address(String line,String city,String pincode){
        this.line=line;
        this.city=city;
        this.pincode=pincode;
        }
        public String getLine(){
        return line;
        }
           public String getCity(){
        return city;
        }
           public String getPincode(){
        return pincode;
        }
}
public String getName()
{
return name;
}     public Address getAddress(){
        return address;
        }
           @Override
public Customer clone() {
      try {
```

```
            return (Customer)super.clone();
        } catch(CloneNotSupportedException e) {
            throw new AssertionError();
        }
}
}

public class prac3 {

    public static void main(String[] args) {
        Account a1=new Account("101","Krish",5000);
        Account a2=new Account("101","Rahul",9000);
        Account a3=new Account("102","Amit",7000);

        System.out.println(a1);
        System.out.println(a2);
        System.out.println(a3);
        System.out.println("a1 equals a2: " +a1.equals(a2));
        System.out.println("a1 equals a3: " +a1.equals(a3));

        Object obj=a1;

        if (obj instanceof Account) {
            System.out.println("obj is an Account");
        }
        Customer.Address address =
            new Customer.Address(
```

```
                "123 Main Road",
                "Anand",
                "388001"
        );
        Customer c1 = new Customer("Krish",address);


        Customer c2 =c1.clone();
        System.out.println("Original Customer: "+c1.getName());
        System.out.println("Cloned Customer: " +c2.getName());
}
}
```

Output :



Key Questions and Solution :

1.Why must equals() and hashCode() be overridden together?

Answer: equals() and hashCode() should be overridden together because Java requires that equal objects must have the same hash code. If only equals() is overridden, collections such as HashMap and HashSet may behave incorrectly when storing or searching for objects.

2.What does the default Object.toString() return, and why override ?

Answer: The default Object.toString() returns a string containing the class name, @ symbol, and the object's hash code in hexadecimal form. We override toString() to provide meaningful and readable information about the object's data, which is useful for debugging and displaying objects.

3.What is the difference between a static nested class and an inner class?

Answer: A static nested class is associated with the outer class and does not require an instance of the outer class to be created. It cannot directly access non-static members of the outer class. An inner class is associated with an instance of the outer class and can directly access both static and non-static members of the outer class.

PRACTICAL – 4

Problem Statement :

1.Password strength (PasswordChecker.java + Driver.java):

(a)In PasswordChecker, check four rules — length ≥ 8, contains an uppercase letter, contains a digit, contains a special character — using regex (e.g. ".*[A-Z].*") or String methods.

(b)Write strength(String pw) that counts how many rules pass and returns "Weak" (0 1), "Medium" (2–3) or "Strong" (4).

(c)In Driver.main: test several passwords; for each, print which rules passed and the final label. Expected output: "abc" → Weak; "Abcd1234!" → Strong.

Aim: To develop a Java program that checks the strength of a password based on four security rules—minimum length, uppercase letter, digit, and special character—and classifies it as **Weak, Medium, or Strong**.

Code :

```
public class PasswordChecker {
public static boolean hasValidLength(String pw) {
    return pw.length() >= 8;
}
public static boolean hasUpperCase(String pw) {
    return pw.matches(".*[A-Z].*");
}


   public static boolean hasDigit(String pw) {
       return pw.matches(".*[0-9].*");
   }
   public static boolean hasSpecialCharacter(String pw) {
    return pw.matches(".*[^a-zA-Z0-9].*");
   }
   public static String strength(String pw) {

       int count = 0;

       if (hasValidLength(pw)) {
           count++;
       }

       if (hasUpperCase(pw)) {
           count++;
       }
```

```
        if (hasDigit(pw)) {
            count++;
        }

        if (hasSpecialCharacter(pw)) {
            count++;
        }

        if (count <= 1) {
            return "Weak";
        }
        else if (count <= 3) {
            return "Medium";
        }
        else {
            return "Strong";
        }
}
}
```

Driver:

```
public class Driver {

    public static void main(String[] args) {

        String[] passwords = {
            "abc",
            "abcdefgh",
```

```
        "Abcd1234",
        "Abcd1234!"
    };

    for (String password : passwords) {

        System.out.println("Password: " + password);
        System.out.println(
            "Length >= 8: "
            + PasswordChecker.hasValidLength(password)
        );
        System.out.println(
            "Uppercase letter: "
            + PasswordChecker.hasUpperCase(password)
        );
        System.out.println(
            "Digit: "
            + PasswordChecker.hasDigit(password)
      }
        System.out.println(
            "Special character: "
            + PasswordChecker.hasSpecialCharacter(password)
        );
        System.out.println(
            "Strength: "
            + PasswordChecker.strength(password)
        );
```

```
            System.out.println(" ------------------- ");
        }
}
}
```

Output :

```
Password: abc
Length >= 8: false
Uppercase letter: false
Digit: false
Special character: false
Strength: Weak
--------------------
Password: abcdefgh
Length >= 8: true
Uppercase letter: false
Digit: false
Special character: false
Strength: Weak
--------------------
Password: Abcd1234
Length >= 8: true
Uppercase letter: true
Digit: true
Special character: false
Strength: Medium
--------------------
Password: Abcd1234!
Length >= 8: true
Uppercase letter: true
Digit: true
Special character: true
Strength: Strong
--------------------
```

Practical-4.2

Problem Statement :

2.Chat-log filter (ChatFilter.java + Driver.java):

(a)In Driver.main, define a String[] of log lines like “10:05 alice Hello there” (include one malformed line with fewer than 3 parts).

(b)Read a keyword from the user.

(c)For each line, split into time/user/message with split(“ ”, 3); skip lines that do not have 3 parts.

(d)If the message contains the keyword (case-insensitive), increment a counter and append “time user: message” to a StringBuilder.

(e)Print “Matches: N” and the built report. Expected output: keyword “hello” over 3 lines (1 malformed) → “Matches: 1” and the matching line.

Aim: To develop a Java program that processes chat log entries, filters messages based on a user-provided keyword, handles malformed log lines, and displays the number of matching messages along with their details.

Code :

ChatFilter:

```
class ChatFilter {


    public static String filter(String[] logs, String keyword) {


        int count = 0;
        StringBuilder report = new StringBuilder();
        for (String line : logs) {
            String[] parts = line.split(" ", 3);


            if (parts.length < 3) {
                continue;
            }


            String time = parts[0];
            String user = parts[1];
            String message = parts[2];
            if (message.toLowerCase()
```

```
                .contains(keyword.toLowerCase())) {

            count++;

        report.append(time)
                .append(" ")
                .append(user)
                .append(": ")
                .append(message)
                .append("\n");
        }
    }

    return "Matches: " + count + "\n" + report;
}
}
```

Driver:

```
import java.util.Scanner;
public class Driver {

   public static void main(String[] args) {
      String[] logs = {
            "10:05 alice Hello there",
            "10:06 bob How are you?",
            "10:07 charlie Good morning",
            "10:08",
            "10:09 david Have a nice day"
```

```
        };

        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter keyword: ");
        String keyword = scanner.nextLine();
        String result =
        ChatFilter.filter(logs, keyword);
        System.out.println();
        System.out.println(result);

        scanner.close();
}
}
```

Output :

```
Enter keyword: hello

Matches: 1
10:05 alice: Hello there
```

MINIBANK-4 Problem Statement :

Your classes exist, but the bank must never trust raw user input. In this lab you build MiniBank's validation and command-parsing layer — regex checks for mobile, email, PAN and IFSC, and a parser that turns a typed line such as "DEPOSIT AC0001 500" into a Command object. Practical 14 will run every menu action through this layer.

1.Create a class named Validator with public static methods that use compiled regular expressions (Pattern): isValidMobile(String), isValidEmail(String), isValidPan(String) and isValidIfsc(String); each returns a boolean.

2.Define an enum named TransactionType with the constants DEPOSIT, WITHDRAW and TRANSFER. Then define a record named Command with three fields: a TransactionType type, a String accountNumber and a long amount.

3.Create a class named CommandParser with a public static method parse(String line) that splits a line such as "DEPOSIT AC0001 500" with split(), converts the parts, and returns a Command object.

4.Create a class named StatementFormatter with a public static method buildStatement(Account account) that uses a StringBuilder to assemble a multi-line account statement string.

5.In main, test each validator with one correct and one wrong input, then parse a sample command and print its three parts.

Code :

```
import java.util.regex.Pattern;

class Validator {

    private static final Pattern MOBILE_PATTERN =
            Pattern.compile("^[6-9][0-9]{9}$");

    private static final Pattern EMAIL_PATTERN =
            Pattern.compile("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+$");
```

```
    private static final Pattern PAN_PATTERN =
            Pattern.compile("^[A-Z]{5}[0-9]{4}[A-Z]$");

    private static final Pattern IFSC_PATTERN =
            Pattern.compile("^[A-Z]{4}0[A-Z0-9]{6}$");

    public static boolean isValidMobile(String mobile) {
        return mobile != null
                && MOBILE_PATTERN.matcher(mobile).matches();
    }

    public static boolean isValidEmail(String email) {
        return email != null
                && EMAIL_PATTERN.matcher(email).matches();
    }

    public static boolean isValidPan(String pan) {
        return pan != null
                && PAN_PATTERN.matcher(pan).matches();
    }

public static boolean isValidIfsc(String ifsc) {
        return ifsc != null
                && IFSC_PATTERN.matcher(ifsc).matches();
}
}
enum TransactionType {
```

```
DEPOSIT,
WITHDRAW,
TRANSFER
}
record Command(
        TransactionType type,
        String accountNumber,
        long amount
) {
}
class CommandParser {

    public static Command parse(String line) {

        String[] parts = line.trim().split("\\s+");

        TransactionType type =
        TransactionType.valueOf(parts[0]);

        String accountNumber = parts[1];

        long amount =
        Long.parseLong(parts[2]);

        return new Command(
                type,
                accountNumber,
```

```
            amount
        );
}
}
class Account {

    private String accountNumber;
    private String holderName;
    private long balance;

    public Account(
            String accountNumber,
            String holderName,
            long balance) {

        this.accountNumber = accountNumber;
        this.holderName = holderName;
        this.balance = balance;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public String getHolderName() {
        return holderName;
    }
```

```
public long getBalance() {
        return balance;
}
}
class StatementFormatter {

    public static String buildStatement(Account account) {

        StringBuilder statement = new StringBuilder();

        statement.append("----- Account Statement ----- \n");

        statement.append("Account Number: ")
                .append(account.getAccountNumber())
        .append("\n");

        statement.append("Holder Name: ")
                .append(account.getHolderName())
                .append("\n");

        statement.append("Balance: ")
                .append(account.getBalance())
                .append("\n");

        statement.append("                                    ");
```

```
        return statement.toString();
}
}
public class minibank4 {

    public static void main(String[] args) {
        System.out.println("===== VALIDATOR TESTS =====");
        System.out.println("\nMobile:");

        System.out.println(
        "9876543210 -> "
        + Validator.isValidMobile("9876543210") );

        System.out.println(
        "12345 -> "
        + Validator.isValidMobile("12345") );
        System.out.println("\nEmail:");

        System.out.println(
        "user@gmail.com -> "
        + Validator.isValidEmail("user@gmail.com") );

        System.out.println(
        "user@gmail -> "
```

```
        + Validator.isValidEmail("user@gmail")
);
System.out.println("\nPAN:");

System.out.println(
"ABCDE1234F -> "
+ Validator.isValidPan("ABCDE1234F") );

System.out.println(
"ABC1234 -> "
+ Validator.isValidPan("ABC1234") );
System.out.println("\nIFSC:");

System.out.println(
"SBIN0001234 -> "
+ Validator.isValidIfsc("SBIN0001234") );

System.out.println(
"SBIN1234 -> "
+ Validator.isValidIfsc("SBIN1234") );

System.out.println("\n===== COMMAND PARSER =====");
```

```
String line = "DEPOSIT AC0001 500";

Command command =
CommandParser.parse(line);

System.out.println(
"Type: " + command.type()
);

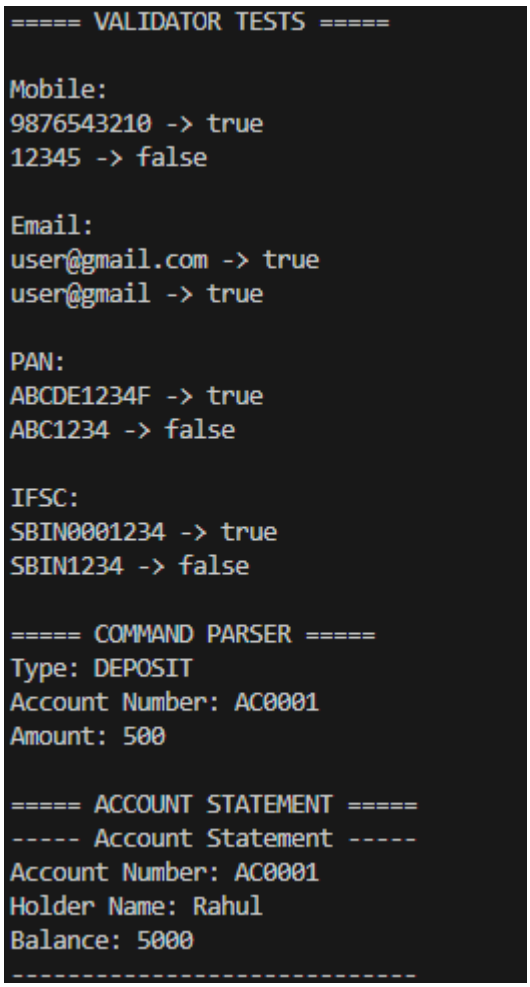
System.out.println(
"Account Number: "
+ command.accountNumber()
);

System.out.println(
"Amount: " + command.amount()
);
System.out.println(
"\n===== ACCOUNT STATEMENT =====" );

Account account =
new Account(
        "AC0001",
        "Rahul",
        5000
);
```

```
        System.out.println(
                StatementFormatter.buildStatement(account)
        );
    }
}
```

Output :



```
===== VALIDATOR TESTS =====

Mobile:
9876543210 -> true
12345 -> false

Email:
user@gmail.com -> true
user@gmail -> true

PAN:
ABCDE1234F -> true
ABC1234 -> false

IFSC:
SBIN0001234 -> true
SBIN1234 -> false

===== COMMAND PARSER =====
Type: DEPOSIT
Account Number: AC0001
Amount: 500

===== ACCOUNT STATEMENT =====
----- Account Statement -----
Account Number: AC0001
Holder Name: Rahul
Balance: 5000
-----------------------------
```

Key Questions and Solution :

1.What is the difference between String, StringBuilder and StringBuffer, and when is each preferred?

Answer: String is immutable, meaning its value cannot be changed after creation. It is preferred when the text does not need frequent modification. StringBuilder is mutable and is preferred when many string modifications are required, especially in single-threaded programs. StringBuffer is also mutable, but its methods are synchronized, making it safer for use in multithreaded programs. It is preferred when thread safety is required.

2.What does the regular expression [6-9][0-9]{9} match, and why use anchors?

Answer: The regular expression [6-9][0-9]{9} matches a 10-digit number where the first digit is between 6 and 9, such as 9876543210. Anchors such as ^ and $ are used to ensure that the entire input matches the pattern rather than only a part of the input. Example: ^[6-9][0-9]{9}$ Here, ^ represents the beginning and $ represents the end of the input.

3.Why should input be validated at the boundary before it is used?

Answer: Input should be validated at the boundary to ensure that only correct, valid, and safe data enters the application. It prevents invalid values from causing errors or unexpected behavior later. It also makes the program more reliable and secure by rejecting incorrect input early.

PRACTICAL – 5

Problem Statement :

Work these in your lab-NN/ folder before the project: do the first one or two in the lab; the last (For Advanced Learners) is for home.

1.Shape areas: an abstract Shape with abstract area(); subclasses Circle, Rectangle,
Triangle. Put a mix in a Shape[] and loop once to print each area and the running total,
tracking the largest — one loop handling every shape through polymorphism.

2.Payroll: an abstract Employee with abstract monthlySalary(); FullTime (fixed), PartTime (hours × rate) and Intern (stipend), each calling super for the shared name/id. Loop a mixed array to print each salary and the total; use instanceof to add a note only for interns.

3.(For Advanced Learners – home) Model a few media types that each compute a late fee differently and total the fees for a returned batch. Design the hierarchy and rules yourself.

Code :

```
abstract class Shape {
abstract double area();
}

class Circle extends Shape {
 double radius;
Circle(double radius) {
        this.radius=radius;
 }
double area() {
```

```
        return Math.PI*radius*radius;
}
}

class Rectangle extends Shape {
double length,width;
Rectangle(double length, double width) {
        this.length=length;
        this.width=width;
}
double area() {
        return length*width;
}
}
class Triangle extends Shape {
double base,height;
double base,height;
Triangle(double base,double height) {
        this.base=base;
        this.height=height;
}
double area() {
        return 0.5*base*height;
}
}
public class prac5_1 {
public static void main(String[] args) {
```

```
        Shape[] shapes={
            new Circle(120),
            new Rectangle(130, 6),
            new Triangle(104, 3)
        };
        double total=0;
        double largest=0;
        for (Shape s:shapes) {
            double a=s.area();
            System.out.println("Area = " + a);
            total+=a;
            if (a>largest) {
                largest = a;
            }
        }
        System.out.println("Total Area = "+total);
    System.out.println("Largest Area = "+largest); }
}
```

Output :

```
PS C:\Users\Admin\OneDrive\Desktop\java> cd "c:\Users\Admin\OneDrive\Desktop\java\" ; if ($?) { javac prac5_1.java } ; if ($?) { java prac5_1 }
Area = 452.3893421169302
Area = 78.0
Area = 21.0
Total Area = 551.3893421169303
Largest Area = 452.3893421169302
PS C:\Users\Admin\OneDrive\Desktop\java>
```

MINIBANK-5 Problem Statement :

A real bank has different account types. Building on the single Account class, this lab makes Account abstract and adds three subclasses — SavingsAccount, CurrentAccount and FixedDepositAccount — each with its own interest and withdrawal rules chosen automatically by polymorphism. This lets the rest of MiniBank treat every account through one common type.

1.Make Account an abstract class. Add two abstract methods: double interestRate() and boolean canWithdraw(long amount).

2.Create a class SavingsAccount that extends Account with a long field minBalance; interestRate() returns 4.0; canWithdraw() returns true only if the balance stays greater than or equal to minBalance.

3.Create a class CurrentAccount that extends Account with a long field overdraftLimit; interestRate() returns 0; canWithdraw() allows the balance to fall to minus overdraftLimit.

4.Create a class FixedDepositAccount that extends Account; interestRate() returns 7.0; canWithdraw() returns false (the deposit is locked).

5.Call super(...) from each subclass constructor to initialise the inherited fields.

6.In main, place objects of the three types in an Account[] array, loop through it and call interestRate() on each (this demonstrates run-time polymorphism); use a pattern instanceof check to handle one type specially.

Code :

```
abstract class Account{
    long accountNumber;
    String holderName;
    long balance;

    Account(long accountNumber,String holderName,long balance){
        this.accountNumber=accountNumber;
        this.holderName=holderName;
        this.balance=balance;
```

```
    }

abstract double interestRate();
abstract boolean canWithdraw(long amount); }

class SavingsAccount extends Account{
long minBalance;

SavingsAccount(long accountNumber,String holderName,long balance,long
minBalance){
        super(accountNumber,holderName,balance);
        this.minBalance=minBalance;
}

    double interestRate(){
        return 4.0;
    }

boolean canWithdraw(long amount){
    return balance-amount>=minBalance; }
}

class CurrentAccount extends Account{
long overdraftLimit;
```

```
CurrentAccount(long accountNumber,String holderName,long balance,long
overdraftLimit){
        super(accountNumber,holderName,balance);
        this.overdraftLimit=overdraftLimit;
}

    double interestRate(){
        return 0;
    }

   boolean canWithdraw(long amount){
        return balance-amount>=-overdraftLimit;
   }
}
class FixedDepositAccount extends Account{
FixedDepositAccount(long accountNumber,String holderName,long
balance){
        super(accountNumber,holderName,balance);
   }

    double interestRate(){
        return 7.0;
    }

boolean canWithdraw(long amount){
        return false;
}
}
```

```
public class prac5{
  public static void main(String[]args){
        Account[]accounts={
            new SavingsAccount(101,"Krish",50000,10000),
        new CurrentAccount(102,"Rahul",20000,15000),
        new FixedDepositAccount(103,"Amit",100000)
    };
  for(Account account:accounts){
            System.out.println(account.interestRate());
            if(account instanceof FixedDepositAccount fd){
            System.out.println("Fixed Deposit");
            }
        }
  }
}
```

Output :

```
PS C:\Users\Admin\OneDrive\Desktop\MINI BANK> cd "c:\Users\Admin\OneDrive\Desktop\java\" ; if ($?) { javac prac5.java } ; if ($?) { java prac5 }
4.0
0.0
7.0
Fixed Deposit
PS C:\Users\Admin\OneDrive\Desktop\java>
```

Key Questions and Solution :

1.What is the difference between an abstract class and a concrete class?

Answer: An abstract class is a class declared with the abstract keyword. It cannot be instantiated directly and may contain abstract methods that must be implemented by subclasses. A concrete class is a normal class that can be

instantiated directly. It provides implementations for all its required methods.

2.What is dynamic method dispatch (run-time polymorphism)?

Answer: Dynamic method dispatch is the process in which the method to be executed is decided at run time based on the actual object, not the reference type. It occurs when a subclass overrides a method of its parent class. Example: Animal a = new Dog(); a.sound(); Here, the Dog version of sound() is called at run time.

3.What does the super keyword do in a subclass constructor?

Answer: The super keyword is used to call the constructor of the parent class from the subclass constructor. It is generally used to initialize the inherited fields of the parent class. Example: class Dog extends Animal { Dog() { super(); } } Here, super() calls the constructor of the Animal class.

PRACTICAL – 6

Problem Statement:

1.Remote control: an interface Switchable with on()/off() and a default toggle(); Fan and Light implement it. Loop over a Switchable[] toggling each. Then add a functional interface that decides, given a device and an hour, whether it may switch on, and supply it once as an anonymous class and once as a lambda.

Code:

```
interface Switchable {
    void on();
    void off();

    default void toggle() {
        off();
        on();
    }
}

class Fan implements Switchable {
    public void on() {
        System.out.println("Fan ON");
    }

    public void off() {
        System.out.println("Fan OFF");
    }
}
```

```
class Light implements Switchable {
    public void on() {
        System.out.println("Light ON");
    }

    public void off() {
        System.out.println("Light OFF");
    }
}

interface SwitchRule {
    boolean maySwitchOn(Switchable device, int hour);
}

public class RemoteControl {
    public static void main(String[] args) {
        Switchable[] devices = {
        new Fan(),
        new Light()
        };

        for (Switchable device : devices) {
        device.toggle();
        }

        SwitchRule rule1 = new SwitchRule() {
        public boolean maySwitchOn(Switchable device, int hour) {
```

```
            return hour >= 6 && hour <= 22;
        }
        };

        SwitchRule rule2 = (device, hour) -> hour >= 8 && hour <= 20;

    System.out.println("Anonymous Rule: " + rule1.maySwitchOn(devices[0],
10));
    System.out.println("Lambda Rule: " + rule2.maySwitchOn(devices[1], 23));
    }
}
```

Output/SS:

```
Fan OFF
Fan ON
Light OFF
Light ON
Anonymous Rule: true
Lambda Rule: false
```

Problem Statement:

2.Notification senders: a functional interface Notifier with send(String); make email and SMS senders as lambdas, hold them in an array, and broadcast one message through all. Add a marker interface Urgent and send twice for senders marked urgent.

Code:

```
@FunctionalInterface
interface Notifier {
    void send(String message);
}

interface Urgent {
}

class UrgentEmail implements Notifier, Urgent {
    public void send(String message) {
        System.out.println("Email: " + message);
    }
}

public class NotificationDemo {
    public static void main(String[] args) {
        Notifier email = message -> System.out.println("Email: " + message);
        Notifier sms = message -> System.out.println("SMS: " + message);

        Notifier[] senders = {
        email,
```

```
        sms,
        new UrgentEmail()
        };

        String message = "MiniBank transaction successful";

        for (Notifier sender : senders) {
        sender.send(message);

        if (sender instanceof Urgent) {
                sender.send(message);
        }
        }
    }
}
```

Output:

```
Email: MiniBank transaction successful
SMS: MiniBank transaction successful
Email: MiniBank transaction successful
Email: MiniBank transaction successful
```

Part B — Project Work :MiniBank

With several classes now in place, this lab gives MiniBank a clean structure. You express account capabilities as interfaces, organise the code into model, util and service packages, and build a runnable JAR — the form in which the finished application will actually be delivered and run.

1. Create an interface named Transactable with methods void deposit(long amount) and boolean withdraw(long amount); make Account implement it.

2.Create an interface named InterestBearing with a default method double yearlyInterest() that uses interestRate() and the balance.

3.Create a functional interface named WithdrawRule with a single method Boolean allow(Account account, long amount). Use it two ways in main: first with an anonymous class, then with a lambda expression.

4.Create a marker interface named Premium (an interface with no methods).

5.Move all classes into the packages model, util and service (add package statements and fix the import statements). Use one static import.

6.Build a runnable JAR file with a manifest whose Main-Class is the MiniBank class, and run it with the command java -jar.

Code:

```
interface Transactable {
    void deposit(long amount);
    boolean withdraw(long amount);
}

 interface InterestBearing {
     double interestRate();

    default double yearlyInterest() {
        return interestRate() * 0.01 * ((Account) this).getBalance();
    }
}

interface WithdrawRule {
    boolean allow(Account account, long amount);
}
```

```
interface Premium {
}

abstract class Account implements Transactable, InterestBearing {
    private String accountNumber;
    private String holderName;
    private long balance;

    Account(String accountNumber, String holderName, long balance) {
        this.accountNumber = accountNumber;
        this.holderName = holderName;
        this.balance = balance;
    }

    public long getBalance() {
        return balance;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public void deposit(long amount) {
        balance += amount;
    }

    public boolean withdraw(long amount) {
```

```
        if (canWithdraw(amount)) {
        balance -= amount;
        return true;
        }
        return false;
    }

    abstract boolean canWithdraw(long amount);
}

class SavingsAccount extends Account {
    private long minBalance;

    SavingsAccount(String accountNumber, String holderName, long balance,
long minBalance) {
        super(accountNumber, holderName, balance);
        this.minBalance = minBalance;
    }

    public double interestRate() {
        return 4.0;
    }

    boolean canWithdraw(long amount) {
        return getBalance() - amount >= minBalance;
    }
}
```

```
class CurrentAccount extends Account implements Premium {
    private long overdraftLimit;

    CurrentAccount(String accountNumber, String holderName, long balance,
long overdraftLimit) {
        super(accountNumber, holderName, balance);
        this.overdraftLimit = overdraftLimit;
    }

    public double interestRate() {
        return 0.0;
    }

    boolean canWithdraw(long amount) {
        return getBalance() - amount >= -overdraftLimit;
    }
}

class FixedDepositAccount extends Account {
    FixedDepositAccount(String accountNumber, String holderName, long
balance) {
        super(accountNumber, holderName, balance);
    }

    public double interestRate() {
        return 7.0;
    }
    boolean canWithdraw(long amount) {
```

```
        return false;
    }
}

public class MiniBank {
    public static void main(String[] args) {

        Account[] accounts = {
        new SavingsAccount("SA001", "Het", 10000, 2000),
        new CurrentAccount("CA001", "Rahul", 5000, 3000),
        new FixedDepositAccount("FD001", "Jay", 20000) };

        WithdrawRule rule1 = new WithdrawRule() {
        public boolean allow(Account account, long amount) {
            return account.withdraw(amount);
        }
        };

        WithdrawRule rule2 = (account, amount) -> account.withdraw(amount);

        for (Account account : accounts) {
        System.out.println(account.getAccountNumber());
        System.out.println("Interest Rate: " + account.interestRate() + "%");
        System.out.println("Yearly Interest: " + account.yearlyInterest()); }
```

```
        System.out.println("Anonymous Rule: " + rule1.allow(accounts[0], 500));
        System.out.println("Lambda Rule: " + rule2.allow(accounts[1], 1000));


        accounts[0].deposit(1000);


        System.out.println("Savings Balance: " + accounts[0].getBalance());
        System.out.println("Premium Account: " + (accounts[1] instanceof
Premium));
    }
}
```

Output:

```
SA001
Interest Rate: 4.0%
Yearly Interest: 400.0
CA001
Interest Rate: 0.0%
Yearly Interest: 0.0
FD001
Interest Rate: 7.0%
Yearly Interest: 1400.0000000000002
Anonymous Rule: true
Lambda Rule: true
Savings Balance: 10500
Premium Account: true
```

Key Questions and Solution

1.What is the difference between an interface and an abstract class?

An interface defines a contract that classes implement, while an abstract class can contain abstract methods, concrete methods, fields, and constructors.

2.What is a functional interface, and how does a lambda use it?

A functional interface has exactly one abstract method. A lambda expression provides its implementation in a concise way.

3.Why is a default method useful in an interface?

A default method provides an implementation inside an interface, allowing new methods to be added without requiring every implementing class to implement them.

PRACTICAL – 7

Problem Statement:

1.Form validator: Define @NotBlank and @MaxLength(int) for fields and annotate a SignupForm. Write a checker that uses reflection to read each field's annotations and returns the list of validation errors for a given object.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.*;
import java.util.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface NotBlank {
}

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface MaxLength {
   int value();
}

class SignupForm {
   @NotBlank
   @MaxLength(20)
   String name;

   @NotBlank
```

```
    @MaxLength(30)
    String email;

    SignupForm(String name, String email) {
        this.name = name;
        this.email = email;
    }
}

class FormValidator {
    static List<String> validate(Object obj) {
    List<String> errors = new ArrayList<>();

        for (Field field : obj.getClass().getDeclaredFields()) {
        try {
            field.setAccessible(true);
            String value = (String) field.get(obj);

                if (field.isAnnotationPresent(NotBlank.class)) {
                   if (value == null || value.trim().isEmpty()) {
                       errors.add(field.getName() + " cannot be blank");
                    }
                }

                if (field.isAnnotationPresent(MaxLength.class)) {
                    int max = field.getAnnotation(MaxLength.class).value();
```

```
                if (value != null && value.length() > max) {
                errors.add(field.getName() + " is too long");
                }
            }
        } catch (Exception e) {
            System.out.println(e);
        }
        }

        return errors;
    }

    public static void main(String[] args) {
        SignupForm form = new SignupForm("",
"abcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyz");

        List<String> errors = validate(form);

        for (String error : errors) {
        System.out.println(error);
        }
    }
}
```

Output/SS:

```
name cannot be blank
email is too long
```

Problem Statement:

2.Mini test runner: Define a @Run annotation and mark some no-arg methods of a class with it. Use reflection to find and invoke only the @Run methods and report how many ran — the idea behind how JUnit finds @Test methods.

Code:

```
import java.lang.annotation.*;
import java.lang.reflect.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Run {
}

class MyTests {
    @Run
    public void testOne() {
        System.out.println("Test One");
    }

    @Run
    public void testTwo() {
        System.out.println("Test Two");
    }

    public void normalMethod() {
        System.out.println("Normal Method");
    }
```

```
}

public class MiniTestRunner {
    public static void main(String[] args) throws Exception {
        MyTests obj = new MyTests();
        int count = 0;

        for (Method method : MyTests.class.getDeclaredMethods()) {
        if (method.isAnnotationPresent(Run.class) &&
            method.getParameterCount() == 0) {

                method.invoke(obj);
                count++;
        }
        }
        System.out.println("Tests ran: " + count);
    }
}
```

Output:

```
Test One
Test Two
Tests ran: 2
```

Part B — Project Work :MiniBank

Now that you are comfortable with classes and interfaces, you add metadata to MiniBank. You define custom annotations (@Id, @Positive, @MaxLength) and read them at run time with reflection to validate any object automatically — the

same idea real frameworks use. The @Id marker will later identify each record in the generic repository you build in Practical 13.

1.Add the @Override annotation above every overridden method written so far (toString, equals, hashCode and the subclass methods). Add @FunctionalInterface above the WithdrawRule interface.

2.In a package named model.annotation, define three custom annotations: @Id (a marker, with @Target(FIELD) and @Retention(RUNTIME)); @Positive with a member String message() default "must be > 0"; and @MaxLength with a member int value().

3.Put @Id on the accountNumber field and @Positive on the balance field of Account.

4.Create a class named AnnotationValidator with a public static method String[] validate(Object obj) that uses reflection — getClass().getDeclaredFields(), field.isAnnotationPresent(...), field.setAccessible(true) and field.get(obj) — to check the @Positive and @MaxLength rules and return the error messages as a String[].

5.In main, create an Account with a negative balance and run AnnotationValidator.validate() to print the error.

Code:

```
import java.lang.annotation.*;

import java.lang.reflect.*;

import java.util.*;


@Retention(RetentionPolicy.RUNTIME)

@Target(ElementType.FIELD)

@interface Id {

}


@Retention(RetentionPolicy.RUNTIME)

@Target(ElementType.FIELD)
```

```
@interface Positive {
    String message() default "must be > 0";
}

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface MaxLength {
    int value();
}

@FunctionalInterface
interface WithdrawRule {
    boolean check(double amount);
}

class Account {
    @Id
    @MaxLength(10)
    private String accountNumber;

    @Positive
    private double balance;

    Account(String accountNumber, double balance) {
        this.accountNumber = accountNumber;
        this.balance = balance;
    }
```

```
    @Override
    public String toString() {
        return accountNumber + " " + balance;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj)
        return true;

        if (!(obj instanceof Account))
        return false;

        Account a = (Account) obj;
        return accountNumber.equals(a.accountNumber);
    }

    @Override
    public int hashCode() {
        return accountNumber.hashCode();
    }
}

public class MiniBank {

    public static String[] validate(Object obj) {
```

```
List<String> errors = new ArrayList<>();

for (Field field : obj.getClass().getDeclaredFields()) {
try {
   field.setAccessible(true);

      if (field.isAnnotationPresent(Positive.class)) {
         double value = ((Number) field.get(obj)).doubleValue();

         if (value <= 0) {
         Positive p = field.getAnnotation(Positive.class);
         errors.add(field.getName() + " " + p.message());
         }
      }

      if (field.isAnnotationPresent(MaxLength.class)) {
         String value = (String) field.get(obj);
         int max = field.getAnnotation(MaxLength.class).value();

         if (value.length() > max) {
         errors.add(field.getName() + " length must be <= " + max);
         }
      }

   } catch (Exception e) {
      System.out.println(e);
   }
```

```
        }

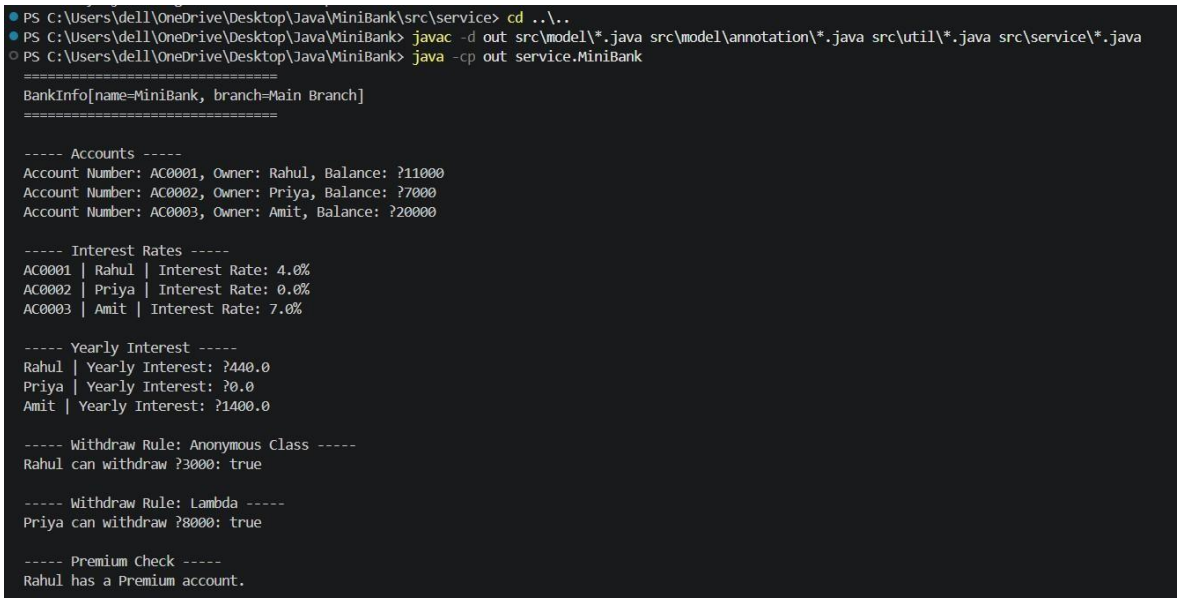
        return errors.toArray(new String[0]);
    }

    public static void main(String[] args) {
        Account account = new Account("ACC123", -500);

        String[] errors = validate(account);

        for (String error : errors) {
            System.out.println(error);
        }
    }
}
```

Output:



```
----- Customer -----
Customer ID: CUST101
Name: Rahul
Email: rahul@gmail.com
Mobile: 9876543210
Address: 123 Main Road, Ahmedabad - 380001
Cloned Customer ID: CUST101
Cloned Customer Name: Rahul

----- Validator Tests -----
Valid Mobile: true
Invalid Mobile: false
Valid Email: true
Invalid Email: true
Valid PAN: true
Invalid PAN: false
Valid IFSC: true
Invalid IFSC: false

----- Command Parser -----
Type: DEPOSIT
Account Number: AC0001
Amount: 500

----- Account Statement -----
----- Account Statement -----
Account Number: AC0001
Owner Name: Rahul
Balance: ?11000
Active: true
```

```
----- Annotation Validation -----
balance: must be > 0

----- instanceof Check -----
The object is an Account.

----- MiniBank Menu -----
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Exit
Enter your choice: 1
Open Account ? to be implemented in a later lab
```

Key Questions and Solution

1.What are the meta-annotations @Retention and @Target used for?

@Retention specifies how long an annotation is retained, while @Target specifies where an annotation can be applied, such as fields, methods, or classes.

2.What is reflection, and what does getDeclaredFields() return?

Reflection allows a Java program to inspect classes and their members at runtime. getDeclaredFields() returns all fields declared in a class, including private fields.

3.How do annotations plus reflection let a framework validate any object?

Annotations define validation rules, and reflection reads those annotations at runtime. The framework checks the object's field values against the rules and reports validation errors.

PRACTICAL – 8

Problem Statement:

1.Guarded calculator: Read two numbers and an operator; throw a custom DivideByZeroException and catch invalid-number input separately, printing a clear message each time; loop until a valid calculation succeeds and use finally to log each attempt.

Code:

```
import java.util.*;

class DivideByZeroException extends Exception {
   DivideByZeroException(String message) {
   super(message);
   }
}

public class GuardedCalculator {

   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      boolean success = false;

      while (!success) {
      try {
         System.out.print("Enter first number: ");
         double a = Double.parseDouble(sc.nextLine());

            System.out.print("Enter operator: ");
```

```
char op = sc.nextLine().charAt(0);

System.out.print("Enter second number: ");
double b = Double.parseDouble(sc.nextLine());

double result = 0;

if (op == '/' && b == 0) {
throw new DivideByZeroException("Cannot divide by zero"); }

switch (op) {
    case '+':
    result = a + b;
    break;
    case '-':
    result = a - b;
    break;
    case '*':
    result = a * b;
    break;
    case '/':
    result = a / b;
    break;
    default:
    System.out.println("Invalid operator");
    continue;
```

```
            }

            System.out.println("Result: " + result);
            success = true;

        } catch (NumberFormatException e) {
            System.out.println("Invalid number input"); }
        catch (DivideByZeroException e) {
            System.out.println(e.getMessage());
        } finally {
            System.out.println("Attempt completed"); }
        }

        sc.close();
    }
}
```

Output/SS:

```
Enter first number: 10
Enter operator: /
Enter second number: 0
Cannot divide by zero
Attempt completed
Enter first number: 20
Enter operator: /
Enter second number: 5
Result: 4.0
Attempt completed
```

Problem Statement:

2.Stock issue: A Warehouse where issue(item, qty) throws a custom checked OutOfStockException carrying the shortfall, and InvalidQuantityException for qty ≤ 0. Process a list of requests, catching and reporting each failure without stopping the run.

Code:

```
import java.util.*;

class OutOfStockException extends Exception {
    int shortfall;

    OutOfStockException(String message, int shortfall) {
        super(message);
        this.shortfall = shortfall;
    }
}

class InvalidQuantityException extends Exception {
    InvalidQuantityException(String message) {
    super(message);
    }
}

class Warehouse {
    private Map<String, Integer> stock = new HashMap<>();

    Warehouse() {
```

```
        stock.put("Pen", 10);
        stock.put("Book", 5);
        stock.put("Pencil", 20);
    }

    void issue(String item, int qty)
    throws OutOfStockException, InvalidQuantityException {

        if (qty <= 0) {
        throw new InvalidQuantityException("Invalid quantity"); }

        int available = stock.getOrDefault(item, 0);

        if (qty > available) {
        int shortfall = qty - available;
        throw new OutOfStockException(
                "Out of stock. Shortfall: " + shortfall, shortfall);
        }

        stock.put(item, available - qty);
        System.out.println("Issued " + qty + " " + item);
    }
}

public class StockIssue {
```

```
    public static void main(String[] args) {
        Warehouse warehouse = new Warehouse();

        String[][] requests = {
        {"Pen", "3"},
        {"Book", "10"},
        {"Pencil", "0"},
        {"Pencil", "5"},
        {"Laptop", "2"}
        };

        for (String[] request : requests) {
        try {
                warehouse.issue(request[0], Integer.parseInt(request[1]));
        } catch (OutOfStockException e) {
                System.out.println(request[0] + ": " + e.getMessage());
        } catch (InvalidQuantityException e) {
                System.out.println(request[0] + ": " + e.getMessage());
        }
        }
    }
}
```

Output:

```
Issued 3 Pen
Book: Out of stock. Shortfall: 5
Pencil: Invalid quantity
Issued 5 Pencil
Laptop: Out of stock. Shortfall: 2
```

Part B — Project Work :MiniBank

Until now a failed withdrawal just returned false. In this lab you make MiniBank fault- tolerant: you replace those flags with a custom exception hierarchy so every failure is explicit and carries information (such as the shortfall). This is the Milestone 1 checkpoint, where the model, validation, annotations and error handling come together.

1.In a package named exception, create a checked base class BankException that extends Exception. Then create InsufficientFundsException, AccountNotFoundException and InvalidAmountException, each extending BankException. Give InsufficientFundsException a long field shortfall.

2.Change withdraw(long amount) to declare throws InsufficientFundsException, InvalidAmountException; throw InsufficientFundsException when the balance is too low and InvalidAmountException when the amount is less than or equal to zero. Make deposit(long amount) throw InvalidAmountException for a non-positive amount.

3.Add a method transfer(Account to, long amount) that uses try, catch and finally and re- throws a BankException when needed.

4.In main, wrap the operations in try-catch blocks and print friendly messages; use a finally block; and show one try-with-resources example using a small AutoCloseable class that you create.

Code:

```
class BankException extends Exception {
    BankException(String message) {
        super(message);
    }
}

class InsufficientFundsException extends BankException {
    long shortfall;
```

```
    InsufficientFundsException(String message, long shortfall) {
        super(message);
        this.shortfall = shortfall;
    }
}

class AccountNotFoundException extends BankException {
    AccountNotFoundException(String message) {
        super(message);
    }
}

class InvalidAmountException extends BankException {
    InvalidAmountException(String message) {
        super(message);
    }
}

class Account {
    private String accountNumber;
    private long balance;

    Account(String accountNumber, long balance) {
        this.accountNumber = accountNumber;
        this.balance = balance;
    }
```

```
    void withdraw(long amount)
    throws InsufficientFundsException, InvalidAmountException {

        if (amount <= 0) {
        throw new InvalidAmountException("Amount must be greater than
zero");
        }

        if (amount > balance) {
        throw new InsufficientFundsException(
                "Insufficient funds. Shortfall: " + (amount - balance),
                amount - balance);
        }

        balance -= amount;
        System.out.println("Withdraw successful: " + amount);
    }

    void deposit(long amount) throws InvalidAmountException {
        if (amount <= 0) {
        throw new InvalidAmountException("Deposit amount must be greater
than zero");
        }

        balance += amount;
        System.out.println("Deposit successful: " + amount);
    }
```

```
    void transfer(Account to, long amount) throws BankException {
        try {
        withdraw(amount);
        to.deposit(amount);
        System.out.println("Transfer successful");
        } catch (BankException e) {
        throw e;
        } finally {
        System.out.println("Transfer completed");
        }
    }

    void display() {
        System.out.println(accountNumber + " Balance: " + balance);
    }
}

class BankResource implements AutoCloseable {
    public void open() {
        System.out.println("Resource opened");
    }

    public void close() {
        System.out.println("Resource closed");
    }
}
```

```
public class MiniBank {
    public static void main(String[] args) {

        Account a1 = new Account("A101", 1000);
        Account a2 = new Account("A102", 500);

        try {
        a1.withdraw(1500);
        } catch (InsufficientFundsException e) {
        System.out.println(e.getMessage());
        } catch (InvalidAmountException e) {
        System.out.println(e.getMessage());
        } finally {
        System.out.println("Withdrawal operation completed"); }

        try {
        a1.deposit(-100);
        } catch (InvalidAmountException e) {
        System.out.println(e.getMessage());
        } finally {
        System.out.println("Deposit operation completed"); }

        try {
        a1.transfer(a2, 300);
        } catch (BankException e) {
```

```
        System.out.println(e.getMessage());
        }

        try (BankResource resource = new BankResource()) {
        resource.open();
        System.out.println("Using bank resource");
        }

        a1.display();
        a2.display();
    }
}
```

Output:

```
Insufficient funds. Shortfall: 500
Withdrawal operation completed
Deposit amount must be greater than zero
Deposit operation completed
Withdraw successful: 300
Deposit successful: 300
Transfer successful
Transfer completed
Resource opened
Using bank resource
Resource closed
A101 Balance: 700
A102 Balance: 800
```

Key Questions and Solution

1.What is the difference between a checked and an unchecked exception?

Checked exceptions are checked at compile time and must be caught or declared, while unchecked exceptions occur at runtime and do not require mandatory handling.

2.What is the purpose of the finally block, and when does it run?

The finally block is used for cleanup operations and normally executes whether an exception occurs or not.

3.How does try-with-resources guarantee a resource is closed?

Try-with-resources automatically closes resources that implement AutoCloseable when the try block finishes, even if an exception occurs.

PRACTICAL – 9

Problem Statement:

1.Counter race: Several threads each increment a shared counter many times. Run it without synchronization to watch the total come out too low, then make the increment synchronized and confirm it is exactly right every run.

Code:

```
class Counter {
    int count = 0;

    void increment() {
        count++;
    }

    synchronized void synchronizedIncrement() {
        count++;
    }
}

public class CounterRace {
    public static void main(String[] args) throws Exception {

         Counter counter1 = new Counter();
         Thread[] threads = new Thread[10];

        for (int i = 0; i < 10; i++) {
        threads[i] = new Thread(() -> {
            for (int j = 0; j < 10000; j++) {
```

```
            counter1.increment();
        }
    });
    threads[i].start();
    }

    for (Thread t : threads) {
    t.join();
    }

    System.out.println("Without synchronization: " + counter1.count);

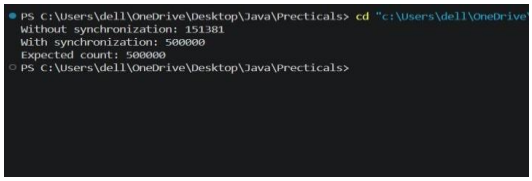
    Counter counter2 = new Counter();
    Thread[] threads2 = new Thread[10];

    for (int i = 0; i < 10; i++) {
    threads2[i] = new Thread(() -> {
        for (int j = 0; j < 10000; j++) {
            counter2.synchronizedIncrement();
        }
    });
    threads2[i].start();
    }

    for (Thread t : threads2) {
    t.join();
    }
```

```
        System.out.println("With synchronization: " + counter2.count);
    }
}
```

Output/SS:



Problem Statement:

2.Seat booking race: A shared seatsLeft of 5 with about 10 threads each trying to book one seat. Without synchronization show that more than 5 can succeed (oversell); with a synchronized book() exactly 5 do.

Code:

```
class Cinema {
    int seatsLeft;

    Cinema(int seats) {
        seatsLeft = seats;
    }

    void book() {
        if (seatsLeft > 0) {
        try {
            Thread.sleep(10);
        } catch (Exception e) {
        }

        seatsLeft--;
        System.out.println(Thread.currentThread().getName() + " booked");
        }
    }

    synchronized void synchronizedBook() {
        if (seatsLeft > 0) {
        seatsLeft--;
```

```
        System.out.println(Thread.currentThread().getName() + " booked");
    }
    }
}

public class SeatBooking {
    public static void main(String[] args) throws Exception {

        Cinema cinema1 = new Cinema(5);
        Thread[] threads = new Thread[10];

        for (int i = 0; i < 10; i++) {
        threads[i] = new Thread(cinema1::book, "Thread-" + (i + 1));
        threads[i].start();
        }

        for (Thread t : threads) {
        t.join();
        }

        System.out.println("Without synchronization - Seats left: " +
cinema1.seatsLeft);

        Cinema cinema2 = new Cinema(5);
        Thread[] threads2 = new Thread[10];

        for (int i = 0; i < 10; i++) {
```

```
threads2[i] = new Thread(cinema2::synchronizedBook, "Thread-" + (i + 1));
            threads2[i].start();
        }
        for (Thread t : threads2) {
            t.join();
        }
        System.out.println("With synchronization - Seats left: " +
cinema2.seatsLeft);
    }
}
```

Output:

```
Thread-7 booked
Thread-8 booked
Thread-3 booked
Thread-9 booked
Thread-4 booked
Thread-5 booked
Thread-2 booked
Thread-6 booked
Thread-10 booked
Thread-1 booked
Without synchronization - Seats left: -2
Thread-1 booked
Thread-4 booked
Thread-6 booked
Thread-8 booked
Thread-3 booked
With synchronization - Seats left: 0
```

Part B — Project Work :MiniBank

A real bank serves many users at once. With the single-user logic working, this lab runs many threads against one account, lets you watch the balance come out wrong (a race condition), and then makes deposit and withdraw thread-safe with synchronization — protecting MiniBank's shared data.

1.Create a class named AccountWorker that implements Runnable with fields Account account, int times and long amount; in run(), loop times times and call account.deposit(amount).

2.In main, create one account, start about 10 threads using AccountWorker, call join() on each, then print the final balance — it will be wrong because of the race condition.

3.Make deposit and withdraw synchronized (or use a synchronized block on a private lock object) and run the program again — the balance is now correct.

4.Print the thread names to observe the thread lifecycle.

Code:

```
class Account {
    private long balance;

    Account(long balance) {
        this.balance = balance;
    }

    synchronized void deposit(long amount) {
        long temp = balance;
        temp = temp + amount;
        balance = temp;
    }

    synchronized void withdraw(long amount) {
        long temp = balance;
        temp = temp - amount;
        balance = temp;
    }

    long getBalance() {
```

```
        return balance;
    }
}

class AccountWorker implements Runnable {
    Account account;
    int times;
    long amount;

    AccountWorker(Account account, int times, long amount) {
        this.account = account;
        this.times = times;
        this.amount = amount;
    }

    public void run() {
        System.out.println(Thread.currentThread().getName() + " started");

        for (int i = 0; i < times; i++) {
        account.deposit(amount);
        }

        System.out.println(Thread.currentThread().getName() + " finished");
    }
}

public class MiniBank {
```

```
    public static void main(String[] args) throws Exception {

        Account account = new Account(0);

        Thread[] threads = new Thread[10];

        for (int i = 0; i < 10; i++) {
        threads[i] = new Thread(
            new AccountWorker(account, 1000, 10),
            "Worker-" + (i + 1)
        );

        threads[i].start();
        }

        for (Thread t : threads) {
        t.join();
        }

        System.out.println("Final balance: " + account.getBalance());
    }
}
```

Output:

```
Worker-7 started
Worker-8 started
Worker-4 started
Worker-1 started
Worker-3 started
Worker-2 started
Worker-6 started
Worker-9 started
Worker-10 started
Worker-5 started
Worker-7 finished
Worker-8 finished
Worker-4 finished
Worker-1 finished
Worker-2 finished
Worker-5 finished
Worker-10 finished
Worker-3 finished
Worker-9 finished
Worker-6 finished
Final balance: 100000
```

Key Questions and Solution

1.What is a race condition, and why does it occur here?

A race condition occurs when multiple threads access and modify shared data simultaneously, causing unpredictable results. Here, multiple threads try to book the same available seats at the same time.

2.What does the synchronized keyword do?

The synchronized keyword allows only one thread at a time to execute a synchronized method or block on the same object, preventing race conditions.

3.What are the states in a thread's life cycle?

Java has six thread states: NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING, and TERMINATED.

PRACTICAL – 10

Problem Statement:

1.Thread-pool runner: With a fixed pool of 3 (ExecutorService), submit about 10 Runnable tasks that print their id and sleep briefly; shut down and await completion, observing the few threads being reused.

Code:

```
import java.util.concurrent.*;

public class ThreadPoolRunner {
   public static void main(String[] args) throws Exception {

      ExecutorService pool = Executors.newFixedThreadPool(3);

      for (int i = 1; i <= 10; i++) {
      int id = i;

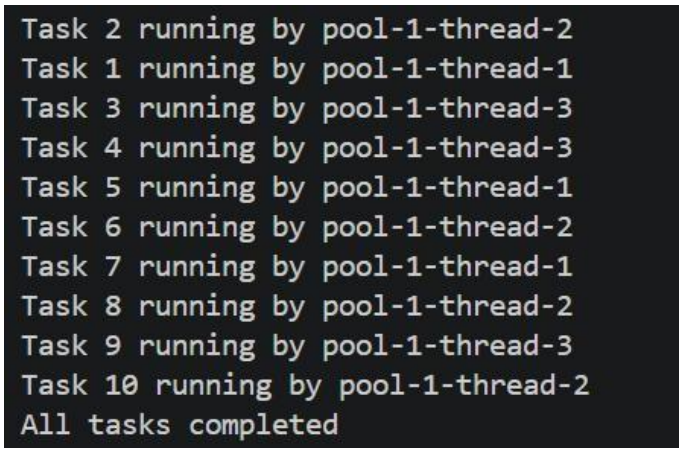
         pool.execute(() -> {
            System.out.println("Task " + id + " running by " +
                  Thread.currentThread().getName());

         try {
            Thread.sleep(500);
            } catch (Exception e) {
         }
      });
      }
```

```
        pool.shutdown();
        pool.awaitTermination(10, TimeUnit.SECONDS);

        System.out.println("All tasks completed");
    }
}
```

Output/SS:



Problem Statement:

2.Producer–consumer: One thread adds numbers to a small shared buffer and another removes them, coordinated with wait()/notify(); produce and consume a fixed number of items in order with none lost.

Code:

```
import java.util.*;

class Buffer {
    Queue<Integer> queue = new LinkedList<>();
    int size = 3;

    synchronized void produce(int value) throws Exception {
        while (queue.size() == size) {
        wait();
        }

        queue.add(value);
        System.out.println("Produced: " + value);
        notify();
    }

    synchronized int consume() throws Exception {
        while (queue.isEmpty()) {
        wait();
        }

        int value = queue.remove();
```

```
        System.out.println("Consumed: " + value);
        notify();

        return value;
    }
}

public class ProducerConsumer {
    public static void main(String[] args) throws Exception {

        Buffer buffer = new Buffer();

        Thread producer = new Thread(() -> {
        try {
            for (int i = 1; i <= 5; i++) {
                buffer.produce(i);
            }
        } catch (Exception e) {
            System.out.println(e);
        }
        });

        Thread consumer = new Thread(() -> {
        try {
            for (int i = 1; i <= 5; i++) {
                buffer.consume();
            }
```

```
            } catch (Exception e) {
                System.out.println(e);
            }
        });

        producer.start();
        consumer.start();

        producer.join();
        consumer.join();

        System.out.println("Completed");
    }
}
```

Output:

```
Produced: 1
Produced: 2
Produced: 3
Consumed: 1
Consumed: 2
Consumed: 3
Produced: 4
Produced: 5
Consumed: 4
Consumed: 5
Completed
```

Part B — Project Work :MiniBank

Building on basic threads, this lab gives MiniBank a proper transaction engine. You process a batch of transactions on a managed thread pool, coordinate two

threads with wait/notify, and learn to avoid deadlock with consistent lock ordering — the way back-office systems process many transactions safely.

1.Create a class named TransactionProcessor that holds an ExecutorService created with Executors.newFixedThreadPool(4). Add a method submit(Runnable task) that calls execute(), and a method stop() that calls shutdown() and awaitTermination().

2.Create Runnable tasks that each perform a deposit or withdrawal; submit many of them, then stop the processor and wait for completion.

3.Build a simple producer-consumer: one thread adds transactions to a shared array buffer and another thread processes them, using wait() and notify() inside synchronized blocks.

4.Create a deadlock: two threads transfer money A to B and B to A, locking the two accounts in opposite order. Then fix it by always locking the account with the smaller accountNumber first.

Code:

```
import java.util.*;

import java.util.concurrent.*;


class Account {

    int accountNumber;

    long balance;


    Account(int accountNumber, long balance) {

        this.accountNumber = accountNumber;

        this.balance = balance;

    }


    synchronized void deposit(long amount) {

        balance += amount;
```

```
        System.out.println(Thread.currentThread().getName() + " deposited " +
amount);
    }

    synchronized void withdraw(long amount) {
        balance -= amount;
        System.out.println(Thread.currentThread().getName() + " withdrew " +
amount);
    }
}

class TransactionProcessor {
    ExecutorService service = Executors.newFixedThreadPool(4);

    void submit(Runnable task) {
        service.execute(task);
    }

    void stop() throws Exception {
        service.shutdown();
        service.awaitTermination(10, TimeUnit.SECONDS);
    }
}

class TransactionBuffer {
    Queue<String> transactions = new LinkedList<>();
    int size = 3;
```

```
    synchronized void add(String transaction) throws Exception {
        while (transactions.size() == size) {
        wait();
        }

        transactions.add(transaction);
        System.out.println("Added: " + transaction);
        notify();
    }

    synchronized String remove() throws Exception {
        while (transactions.isEmpty()) {
        wait();
        }

        String transaction = transactions.remove();
        System.out.println("Processed: " + transaction);
        notify();

        return transaction;
    }
}

public class MiniBank {

    static void transfer(Account a, Account b, long amount) {
```

```
    Account first;
    Account second;

    if (a.accountNumber < b.accountNumber) {
    first = a;
    second = b;
    } else {
    first = b;
    second = a;
    }

    synchronized (first) {
    synchronized (second) {
        a.balance -= amount;
        b.balance += amount;

        System.out.println(
                "Transferred " + amount +
                " from " + a.accountNumber +
                " to " + b.accountNumber
        );
    }
    }
}

public static void main(String[] args) throws Exception {
```

```
Account a = new Account(101, 1000);

TransactionProcessor processor = new TransactionProcessor();

for (int i = 1; i <= 5; i++) {
processor.submit(() -> a.deposit(100)); }

for (int i = 1; i <= 3; i++) {
processor.submit(() -> a.withdraw(50)); }

processor.stop();

System.out.println("Balance after transactions: " + a.balance);

TransactionBuffer buffer = new TransactionBuffer();

Thread producer = new Thread(() -> {
try {
    for (int i = 1; i <= 5; i++) {
            buffer.add("Transaction-" + i);
    }
} catch (Exception e) {
    System.out.println(e);
}
});
```

```
Thread consumer = new Thread(() -> {
try {
    for (int i = 1; i <= 5; i++) {
            buffer.remove();
    }
} catch (Exception e) {
    System.out.println(e);
}
});

producer.start();
consumer.start();

producer.join();
consumer.join();

Account b = new Account(102, 1000);

Thread t1 = new Thread(() -> {
transfer(a, b, 100);
});

Thread t2 = new Thread(() -> {
transfer(b, a, 200);
});
```

```
        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println("Account 101: " + a.balance);
        System.out.println("Account 102: " + b.balance);

        System.out.println("All operations completed");
    }
}
```

Output:

```
pool-1-thread-1 deposited 100
pool-1-thread-2 deposited 100
pool-1-thread-3 deposited 100
pool-1-thread-4 deposited 100
pool-1-thread-4 withdrew 50
pool-1-thread-1 deposited 100
pool-1-thread-2 withdrew 50
pool-1-thread-3 withdrew 50
Balance after transactions: 1350
Added: Transaction-1
Processed: Transaction-1
Added: Transaction-2
Added: Transaction-3
Processed: Transaction-2
Processed: Transaction-3
Added: Transaction-4
Processed: Transaction-4
Added: Transaction-5
Processed: Transaction-5
Transferred 100 from 101 to 102
Transferred 200 from 102 to 101
Account 101: 1450
Account 102: 900
All operations completed
```

Key Questions and Solution

1.Why is a thread pool better than creating a new thread per task?

A thread pool reuses existing threads, reducing thread creation overhead and improving performance and resource management.

2.How do wait() and notify() coordinate two threads?

wait() makes a thread release the object's monitor and wait, while notify() wakes one waiting thread. They must be called while holding the same object's monitor.

3.What causes a deadlock, and how does consistent lock ordering prevent it?

A deadlock occurs when threads wait indefinitely for locks held by each other. Consistent lock ordering prevents circular waiting by ensuring all threads acquire locks in the same order.

PRACTICAL – 11

Problem Statement:

1.Save/load: make a small class Serializable, save an array of its objects to a file with ObjectOutputStream, then read it back into fresh objects and confirm the values survived. Mark one field transient and show it is not saved.

Code:

```
import java.io.*;

class Student implements Serializable {
    private static final long serialVersionUID = 1L;
    private String name;
    private int marks;
    private transient String password;

    Student(String name, int marks, String password) {
    this.name = name;
    this.marks = marks;
    this.password = password;
    }

    public String toString() {
    return name + " | " + marks + " | password = " + password;  }
}

public class prac11_1 {
    public static void main(String[] args) {
    Student[] students = {
            new Student("Krish", 85, "krish@123"),
            new Student("Rahul", 72, "rahul@456"),
            new Student("Amit", 90, "amit@789")
    };

        System.out.println("Original objects :");
        for (Student s : students) {
```

```
            System.out.println(s);
        }

        String file = "students.dat";

        try (ObjectOutputStream out =
                new ObjectOutputStream(new FileOutputStream(file))) {
            out.writeObject(students);
            System.out.println("\nSaved " + students.length + " students to " +
            file);
        } catch (IOException e) {
            System.out.println("Save failed : " + e.getMessage());
        }

    try (ObjectInputStream in =
                new ObjectInputStream(new FileInputStream(file))) {
            Student[] loaded = (Student[]) in.readObject();
            System.out.println("\nLoaded objects :");
            for (Student s : loaded) {
                System.out.println(s);
            }
    } catch (IOException | ClassNotFoundException e) {
            System.out.println("Load failed : " + e.getMessage());      }
    }
}
```

Output/SS:

```
Original objects :
Krish | 85 | password = krish@123
Rahul | 72 | password = rahul@456
Amit | 90 | password = amit@789

Saved 3 students to students.dat

Loaded objects :
Krish | 85 | password = null
Rahul | 72 | password = null
Amit | 90 | password = null
```

Practical 11.2:

Problem Statement:

2.Log analyzer: create a few text files, then use Path/Files to read each line by line and count the total lines and how many contain a keyword across all files; also print each file's size from its attributes.

Code:

```
import java.io.IOException;
import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class prac11_2 {
    public static void main(String[] args) throws IOException {
    Path dir = Paths.get("logs");
    Files.createDirectories(dir);

        Files.write(dir.resolve("server1.log"), List.of(
            "10:00 INFO Server started",
            "10:05 ERROR Disk full",
            "10:10 INFO Request served"));
        Files.write(dir.resolve("server2.log"), List.of(
            "11:00 INFO Backup done",
            "11:15 ERROR Timeout",
            "11:20 ERROR Timeout again",
            "11:30 INFO Idle"));
        Files.write(dir.resolve("server3.log"), List.of(
            "12:00 INFO All fine",
            "12:30 INFO Shutdown"));

        String keyword = "ERROR";
        int totalLines = 0;
        int totalMatches = 0;
```

```
        ArrayList<Path> files = new ArrayList<>();
        try (DirectoryStream<Path> stream = Files.newDirectoryStream(dir,
        "*.log")) {
            for (Path p : stream) {
                files.add(p);
            }
        }
        Collections.sort(files);

        for (Path file : files) {
            int lines = 0;
            int matches = 0;
            for (String line : Files.readAllLines(file)) {
                lines++;
                if (line.contains(keyword)) {
                    matches++;
                }
            }
            BasicFileAttributes attr =
                Files.readAttributes(file, BasicFileAttributes.class);
            System.out.println(file.getFileName() + " -> lines: " + lines
                + ", '" + keyword + "' lines: " + matches
                + ", size: " + attr.size() + " bytes");
            totalLines += lines;
            totalMatches += matches;
        }

        System.out.println("                                    ");
    System.out.println("Total lines : " + totalLines);
    System.out.println("Total matches : " + totalMatches);
    }
}
```

Output/SS:

```
server1.log -> lines: 3, 'ERROR' lines: 1, size: 74 bytes
server2.log -> lines: 4, 'ERROR' lines: 2, size: 85 bytes
server3.log -> lines: 2, 'ERROR' lines: 0, size: 40 bytes
------------------------------
Total lines   : 9
Total matches : 3
```

MINIBANK-11

Problem Statement:

So far MiniBank forgets everything when it closes. In this lab you add the persistence and reporting layer: you make the model Serializable and save/load all accounts to a file, write an append-only transaction log, and then use NIO to walk the logs and produce an end-of-day summary report.

1.Make Account (and its subclasses and Customer) implement Serializable, and add a private static final long serialVersionUID field.

2.Create a class named StatePersister with a public static method void save(Account[] accounts, Path file) that uses ObjectOutputStream inside try-with-resources, and a public static method Account[] load(Path file) that uses ObjectInputStream.

3.Create a class named TransactionLog with a public static method void append(Path file, String line) that uses Files.write(...) with CREATE and APPEND.

4.Create a class named ReportGenerator that walks the logs folder using Files.newDirectoryStream, reads each file line by line with a BufferedReader, and adds up the total deposits and total withdrawals in long variables; also read file details (size, last-modified) using Files.readAttributes, then write the summary to a report file.

5.In main, save the accounts, reload them into a new array and print them to confirm the data survived, then generate and print the report.

Code:

```
import java.io.*;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;
import java.util.ArrayList;
import java.util.Collections;

abstract class Account implements Serializable {
    private static final long serialVersionUID = 1L;
    private static int counter = 1;

    private final String accountNumber;
    private String holderName;
    protected long balance;
```

```
    Account(String holderName, long balance) {
    this.accountNumber = String.format("AC%04d", counter++);
    this.holderName = holderName;
    this.balance = balance;
    }

    public String getAccountNumber() { return accountNumber; }
    public String getHolderName() { return holderName; } public
    long getBalance()                { return balance; }

    public void deposit(long amount) {
    balance += amount;
    }

    public boolean withdraw(long amount) {
    if (canWithdraw(amount)) {
        balance -= amount;
        return true;
    }
    return false;
    }

    abstract double interestRate();
    abstract boolean canWithdraw(long amount);

    public String toString() {
    return accountNumber + " | " + holderName + " | " + balance
        + " | " + getClass().getSimpleName();
    }
}

class SavingsAccount extends Account {
    private static final long serialVersionUID = 1L;
    private long minBalance;
```

```
    SavingsAccount(String holderName, long balance, long minBalance) {
    super(holderName, balance);
    this.minBalance = minBalance;
    }
    double interestRate() { return 4.0; }
    boolean canWithdraw(long amount) { return balance - amount >=
    minBalance; }
}

class CurrentAccount extends Account {
    private static final long serialVersionUID = 1L;
    private long overdraftLimit;

    CurrentAccount(String holderName, long balance, long overdraftLimit) {
    super(holderName, balance);
    this.overdraftLimit = overdraftLimit;
    }
    double interestRate() { return 0; }
    boolean canWithdraw(long amount) { return balance - amount >= -
    overdraftLimit; }
}

class FixedDepositAccount extends Account {
    private static final long serialVersionUID = 1L;

    FixedDepositAccount(String holderName, long balance) {
    super(holderName, balance);
    }
    double interestRate() { return 7.0; }
    boolean canWithdraw(long amount) { return false; } }

class Customer implements Serializable {
    private static final long serialVersionUID = 1L;
    private static long customerCounter = 100;
```

```
    private String name;
    private String email;
    private String mobile;
    private final String customerId;

    Customer(String name, String email, String mobile) {
    this.name = name;
    this.email = email;
    this.mobile = mobile;
    this.customerId = "CUST" + (++customerCounter); }
    public String getName()       { return name; }
    public String getEmail()      { return email; }
    public String getMobile()  { return mobile; }
    public String getCustomerId() { return customerId; }
}

class StatePersister {
    public static void save(Account[] accounts, Path file) throws IOException {
    try (ObjectOutputStream out =
                    new ObjectOutputStream(Files.newOutputStream(file))) {
            out.writeObject(accounts);
    }
    }

    public static Account[] load(Path file)
            throws IOException, ClassNotFoundException {
    try (ObjectInputStream in =
                    new ObjectInputStream(Files.newInputStream(file))) {
            return (Account[]) in.readObject();
    }
    }
}

class TransactionLog {
```

```
    public static void append(Path file, String line) throws IOException {
    Files.write(file,
            (line +
            System.lineSeparator()).getBytes(StandardCharsets.UTF_8),
        StandardOpenOption.CREATE, StandardOpenOption.APPEND);    }
}

class ReportGenerator {
    public static String generate(Path logsDir, Path reportFile) throws
    IOException {
    long totalDeposits = 0;
    long totalWithdrawals = 0;
    StringBuilder report = new StringBuilder();
    report.append("===== END OF DAY REPORT =====\n");

        ArrayList<Path> files = new ArrayList<>();
        try (DirectoryStream<Path> stream =
        Files.newDirectoryStream(logsDir)) {
            for (Path p : stream) {
                files.add(p);
            }
        }
        Collections.sort(files);

        for (Path file : files) {
            BasicFileAttributes attr =
                Files.readAttributes(file, BasicFileAttributes.class);

            if (!file.toString().endsWith(".log") || attr.size() == 0) {
                report.append("Skipped :
                ").append(file.getFileName()).append("\n");
                continue;
            }

            report.append("File    : ").append(file.getFileName())
```

```
                .append(" (").append(attr.size()).append(" bytes, modified ")
                .append(attr.lastModifiedTime()).append(")\n");

            try (BufferedReader br = Files.newBufferedReader(file)) {
                String line;
                while ((line = br.readLine()) != null) {
                    String[] parts = line.trim().split("\\s+");
                    if (parts.length != 3) {
                        continue;
                    }
                    long amount = Long.parseLong(parts[2]);
                    if (parts[0].equals("DEPOSIT")) {
                        totalDeposits += amount;
                    } else if (parts[0].equals("WITHDRAW")) {
                    totalWithdrawals += amount;
                    }
                }
            }
        }

        report.append("                                    \n");
        report.append("Total deposits  :
        ").append(totalDeposits).append("\n");
        report.append("Total withdrawals :
        ").append(totalWithdrawals).append("\n");
        report.append("Net change        : ")
                .append(totalDeposits - totalWithdrawals).append("\n");

    Files.writeString(reportFile, report.toString());
    return report.toString();
    }
}

public class minibank11 {
    public static void main(String[] args) throws Exception {
```

```
Path dataDir = Paths.get("data");
Path logsDir = dataDir.resolve("logs");
Path accountsFile = dataDir.resolve("accounts.dat");
Path reportFile = dataDir.resolve("report.txt");
Files.createDirectories(logsDir);

try (DirectoryStream<Path> old = Files.newDirectoryStream(logsDir))
{
    for (Path p : old) {
        Files.delete(p);
    }
}

Account[] accounts = {
    new SavingsAccount("Krish", 5000, 1000),
    new CurrentAccount("Rahul", 2000, 5000),
    new FixedDepositAccount("Amit", 9000) };

Path day1 = logsDir.resolve("day1.log");
Path day2 = logsDir.resolve("day2.log");

accounts[0].deposit(500);
TransactionLog.append(day1, "DEPOSIT AC0001 500");
if (accounts[1].withdraw(1500)) {
    TransactionLog.append(day1, "WITHDRAW AC0002 1500");
}
if (accounts[0].withdraw(1000)) {
    TransactionLog.append(day2, "WITHDRAW AC0001 1000");
}
accounts[2].deposit(1000);
TransactionLog.append(day2, "DEPOSIT AC0003 1000");

Files.createFile(logsDir.resolve("empty.log"));
Files.writeString(logsDir.resolve("notes.txt"), "not a log");
```

```
            System.out.println("===== SAVE ACCOUNTS =====");
            for (Account a : accounts) {
                System.out.println(a);
            }
            StatePersister.save(accounts, accountsFile);
            System.out.println("Saved to " + accountsFile);

            System.out.println("\n===== LOAD ACCOUNTS INTO NEW
            ARRAY =====");
            Account[] loaded = StatePersister.load(accountsFile);
            for (Account a : loaded) {
                System.out.println(a);
            }

        System.out.println("\n" + ReportGenerator.generate(logsDir,
        reportFile));
        System.out.println("Report written to " + reportFile);
        }
}
```

Output/SS:

```
===== SAVE ACCOUNTS =====
AC0001 | Krish | 4500 | SavingsAccount
AC0002 | Rahul | 500 | CurrentAccount
AC0003 | Amit | 10000 | FixedDepositAccount
Saved to data/accounts.dat

===== LOAD ACCOUNTS INTO NEW ARRAY =====
AC0001 | Krish | 4500 | SavingsAccount
AC0002 | Rahul | 500 | CurrentAccount
AC0003 | Amit | 10000 | FixedDepositAccount

===== END OF DAY REPORT =====
File    : day1.log (40 bytes, modified 2026-10-19T03:50:35.567438324Z)
File    : day2.log (41 bytes, modified 2026-10-19T03:50:35.567438324Z)
Skipped : empty.log
Skipped : notes.txt
----------------------------
Total deposits    : 1500
Total withdrawals : 2500
Net change        : -1000

Report written to data/report.txt
```

Key Questions and Solution

1.What does it mean for a class to be Serializable, and why add serialVersionUID?

Answer: Serializable lets an object be saved as bytes and rebuilt later; serialVersionUID is the class version checked while loading.

2.What is the difference between a byte stream and a character stream?

Answer: Byte stream handles raw bytes (binary data); character stream handles text.

3.How does NIO's Path/Files API differ from the old File class, and how is a directory stream iterated?

Answer: Path/Files is simpler and gives clear errors; a DirectoryStream is read with a for-each loop inside try-with-resources.

PRACTICAL – 12

Problem Statement:

1.Word frequency: split a paragraph into words, count each in a HashMap, and print the words sorted by frequency (highest first) using a Comparator.

Code:

```
import java.util.*;

public class prac12_1 {
    public static void main(String[] args) {
    String paragraph = "Java is fun and Java is powerful. "
        + "Java is object oriented and fun to learn. "
            + "Learn Java and practice Java every day.";

        String[] words = paragraph.toLowerCase().split("\\W+");

        HashMap<String, Integer> freq = new HashMap<>();
        for (String w : words) {
            freq.put(w, freq.getOrDefault(w, 0) + 1);
        }

        ArrayList<Map.Entry<String, Integer>> list =
            new ArrayList<>(freq.entrySet());

        list.sort(new Comparator<Map.Entry<String, Integer>>() {
            public int compare(Map.Entry<String, Integer> a,
                    Map.Entry<String, Integer> b) {              int
        c = b.getValue().compareTo(a.getValue());                if (c
        != 0) {
                    return c;
                }
                return a.getKey().compareTo(b.getKey());
            }
        });

        System.out.println("Word frequency (highest first) :");
```

```
    for (Map.Entry<String, Integer> e : list) {
            System.out.println(e.getKey() + " : " + e.getValue());
    }
    }
}
```

Output/SS:

```
Word frequency (highest first) :
java : 5
and : 3
is : 3
fun : 2
learn : 2
day : 1
every : 1
object : 1
oriented : 1
powerful : 1
practice : 1
to : 1
```

Practical 12.2:

Problem Statement:

2.Unique visitors: from a list of login names with repeats, use a HashSet to count unique users and a LinkedHashSet to keep first-seen order; print the unique count and the first three distinct names.

Code:

```
import java.util.*;

public class prac12_2 {
    public static void main(String[] args) {
    String[] logins = {
            "krish", "rahul", "krish", "amit", "priya",
            "rahul", "dhruvi", "amit", "krish", "priya"
    };

        HashSet<String> unique = new HashSet<>(Arrays.asList(logins));
```

```
        LinkedHashSet<String> ordered = new
        LinkedHashSet<>(Arrays.asList(logins));

        System.out.println("Total logins : " + logins.length);
        System.out.println("Unique users : " + unique.size());

    System.out.println("First three distinct names (first-seen order) :");
    int count = 0;
    for (String name : ordered) {
            System.out.println(name);
            count++;
            if (count == 3) {
                break;
            }
    }
    }
}
```

Output/SS:

```
Total logins  : 10
Unique users  : 5
First three distinct names (first-seen order) :
krish
rahul
amit
```

MINIBANK-12

Problem Statement:

Until now accounts have lived in a plain array, which is hard to search and grow. In this lab you upgrade MiniBank's storage to the Collections Framework — a HashMap keyed by account number, sorting with a Comparator, and a thread-safe map for concurrent access. This is the Milestone 2 checkpoint.

1.Create a class named AccountStore that keeps accounts in a HashMap<String, Account> using the account number as the key. Add methods add(Account account), get(String accountNumber) and getAll().

2.Keep a transaction history in an ArrayList<String>.

3.Add a method sortedByBalance() that returns an ArrayList<Account> sorted with a Comparator.

4.Make Account implement Comparable<Account> by comparing account numbers in compareTo().

5.In the TransactionProcessor, use a ConcurrentHashMap for the shared accounts so it is thread-safe.

6.In main, add several accounts to the store and print them sorted by balance.

Code:

```
import java.util.*;
import java.util.concurrent.*;

class Account implements Comparable<Account> {
    private static int counter = 1;

    private final String accountNumber;
    private String ownerName;
    private long balance;

    Account(String ownerName, long balance) {
    this.accountNumber = String.format("AC%04d", counter++);
    this.ownerName = ownerName;
    this.balance = balance;
    }

    public String getAccountNumber() { return accountNumber; }
    public String getOwnerName()     { return ownerName; }
```

```
    public long getBalance() { return balance; }

    public synchronized void deposit(long amount) {
    balance += amount;
    }

    public synchronized boolean withdraw(long amount) {
    if (amount <= balance) {
        balance -= amount;
        return true;
    }
    return false;
    }

    @Override
    public int compareTo(Account other) {
    return this.accountNumber.compareTo(other.accountNumber); }

    @Override
    public String toString() {
    return accountNumber + " | " + ownerName + " | " + balance;
    }
}

class AccountStore {
    private HashMap<String, Account> accounts = new HashMap<>();
    private ArrayList<String> history = new ArrayList<>();

    public void add(Account account) {
    accounts.put(account.getAccountNumber(), account);
    history.add("ADDED " + account.getAccountNumber()); }

    public Account get(String accountNumber) {
    history.add("LOOKUP " + accountNumber);
```

```
    return accounts.get(accountNumber); }

    public ArrayList<Account> getAll() {
    return new ArrayList<>(accounts.values()); }

    public ArrayList<Account> sortedByBalance() {
    ArrayList<Account> list = new ArrayList<>(accounts.values());
    list.sort(new Comparator<Account>() {
        public int compare(Account a, Account b) {
                return Long.compare(b.getBalance(), a.getBalance());
        }
    });
    return list;
    }

    public ArrayList<Account> balanceAbove(long limit) {
    ArrayList<Account> result = new ArrayList<>(); for
    (Account a : accounts.values()) {
        if (a.getBalance() > limit) {
                result.add(a);
        }
    }
    return result;
    }

    public ArrayList<String> getHistory() {
    return history;
    }
}

class TransactionProcessor {
    private final ConcurrentHashMap<String, Account> accounts =
    new ConcurrentHashMap<>();
```

```
    private final ExecutorService pool = Executors.newFixedThreadPool(4);

    public void addAccount(Account account) {
    accounts.put(account.getAccountNumber(), account); }

    public Account get(String accountNumber) {
    return accounts.get(accountNumber); }

    public void submit(Runnable task) {
    pool.execute(task);
    }

    public void stop() throws InterruptedException {
    pool.shutdown();
    pool.awaitTermination(1, TimeUnit.MINUTES);
    }
}

public class minibank12 {
    public static void main(String[] args) throws InterruptedException {
    AccountStore store = new AccountStore();
    store.add(new Account("Krish", 5000));
    store.add(new Account("Rahul", 2000));
    store.add(new Account("Amit", 9000));

        System.out.println("===== LOOK-UP BY ACCOUNT NUMBER
        =====");
        System.out.println("get(\"AC0002\") -> " + store.get("AC0002"));

        System.out.println("\n===== SORTED BY BALANCE (HIGH TO
        LOW) =====");
        for (Account a : store.sortedByBalance()) {
            System.out.println(a);
        }
```

```
        System.out.println("\n===== BALANCE ABOVE 4000 =====");
        for (Account a : store.balanceAbove(4000)) {
            System.out.println(a);
        }

        System.out.println("\n===== TRANSACTION HISTORY =====");
        for (String h : store.getHistory()) {
            System.out.println(h);
        }

        System.out.println("\n===== THREAD-SAFE BATCH =====");
        TransactionProcessor processor = new TransactionProcessor();
        processor.addAccount(new Account("Priya", 0));
        final Account shared = processor.get("AC0004");

    for (int i = 0; i < 100; i++) {
            processor.submit(new Runnable() {
                public void run() {
                    shared.deposit(10);
                }
            });
    }
    processor.stop();
    System.out.println("100 deposits of 10 each -> balance = "
        + shared.getBalance());
    }
}
```

Output/SS:

```
===== LOOK-UP BY ACCOUNT NUMBER =====
get("AC0002") -> AC0002 | Rahul | 2000

===== SORTED BY BALANCE (HIGH TO LOW) =====
AC0003 | Amit | 9000
AC0001 | Krish | 5000
AC0002 | Rahul | 2000

===== BALANCE ABOVE 4000 =====
AC0001 | Krish | 5000
AC0003 | Amit | 9000

===== TRANSACTION HISTORY =====
ADDED AC0001
ADDED AC0002
ADDED AC0003
LOOKUP AC0002

===== THREAD-SAFE BATCH =====
100 deposits of 10 each -> balance = 1000
```

Key Questions and Solution

1. **What is the difference between a List, a Set and a Map?**
   Answer: List is ordered with duplicates, Set has no duplicates, Map stores unique key-value pairs.

2. **When do you use a HashMap versus a TreeMap?**
   Answer: HashMap is fast but unordered; TreeMap keeps keys sorted but is slower.

3. **What is the difference between Comparable and Comparator?**
   Answer: Comparable is the natural order inside the class (compareTo); Comparator is a separate custom order (compare).

PRACTICAL – 13

Problem Statement:

1.Generic containers: a generic Box<T> with get/set and a Pair<A,B>; use them with different types to show type safety without casts, and write a generic method that returns the first element of any array.

Code:

```
class Box<T> {
    private T value;

    Box(T value) {
    this.value = value;
    }

    T get() {
    return value;
    }

    void set(T value) {
    this.value = value;
    }
}

class Pair<A, B> {
    private A first;
    private B second;

    Pair(A first, B second) {
    this.first = first;
    this.second = second;
    }

    A getFirst() { return first; }
    B getSecond() { return second; }

    public String toString() {
```

```
    return "(" + first + ", " + second + ")";    }
}

public class prac13_1 {

    static <T> T firstElement(T[] array) {
    if (array.length == 0) {
        return null;
    }
    return array[0];
    }

    public static void main(String[] args) {
        Box<Integer> intBox = new Box<>(25);
        int number = intBox.get();
        System.out.println("Integer box : " + number);

        Box<String> strBox = new Box<>("Hello");
        String text = strBox.get();
        System.out.println("String box : " + text);

        strBox.set("Generics");
        System.out.println("After set : " + strBox.get());

        Pair<String, Integer> p1 = new Pair<>("Krish", 85);
        Pair<Integer, Double> p2 = new Pair<>(7, 99.5);
        System.out.println("Pair 1      : " + p1);
        System.out.println("Pair 2      : " + p2);

    String[] names = {"Krish", "Rahul", "Amit"};
    Integer[] marks = {85, 72, 90};
    System.out.println("First name : " + firstElement(names));
    System.out.println("First mark : " + firstElement(marks));    }
}
```

Output/SS:

```
Integer box : 25
String box  : Hello
After set   : Generics
Pair 1      : (Krish, 85)
Pair 2      : (7, 99.5)
First name  : Krish
First mark  : 85
```

Practical 13.2:

Problem Statement:

2.Bounded stats: a generic method <N extends Number> double average(N[] values) that works for Integer[], Double[] and Long[]; note why it refuses non-Number types.

Code:

```
public class prac13_2 {

    static <N extends Number> double average(N[] values) {
    double sum = 0;
    for (N v : values) {
        sum += v.doubleValue();
    }
    return sum / values.length;
    }

    public static void main(String[] args) {
        Integer[] ints = {10, 20, 30, 40};
        Double[] doubles = {1.5, 2.5, 3.5};
        Long[] longs = {100L, 200L, 300L};

        System.out.println("Average of Integer[] : " + average(ints));
        System.out.println("Average of Double[] : " + average(doubles));
        System.out.println("Average of Long[] : " + average(longs));

    }
}
```

Output/SS:

```
Average of Integer[] : 25.0
Average of Double[]  : 2.5
Average of Long[]    : 200.0
```

MINIBANK-13

Problem Statement:

The HashMap store from the previous lab works only for accounts. In this lab you make MiniBank's storage reusable: one generic Repository<T> that stores any type safely and finds each object's key through the @Id annotation by reflection — the repository pattern used in real data-access layers. The service layer in Practical 14 will use it.

1.Create a generic class named Repository<T> that holds a HashMap<String, T>. Add methods T save(T entity), T findById(String id), ArrayList<T> findAll() and boolean delete(String id).

2.Inside save(), find the id by reflection: look for the field marked @Id (from Practical 7) and read its value as the map key.

3.Add a method marked @SafeVarargs called saveAll(T... items) that saves several objects at once.

4.Add one generic method with a bounded type parameter, for example static <N extends Number> double average(N[] values).

5.In main, create a Repository<Account>, save several accounts, fetch one by id, and print all of them.

Code:

```
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.reflect.Field;
import java.util.ArrayList;
import java.util.HashMap;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Id {
}

class Account {
    private static int counter = 1;

    @Id
```

```
    private final String accountNumber;
    private String ownerName;
    private long balance;

    Account(String ownerName, long balance) {
    this.accountNumber = String.format("AC%04d", counter++);
    this.ownerName = ownerName;
    this.balance = balance;
    }

    public String getAccountNumber() { return accountNumber; }
    public long getBalance()                { return balance; }
    @Override
    public String toString() {
    return accountNumber + " | " + ownerName + " | " + balance;
    }
}

class Customer {
    private static long customerCounter = 100;

    @Id
    private final String customerId;
    private String name;
    private String email;

    Customer(String name, String email) {
    this.customerId = "CUST" + (++customerCounter);
    this.name = name;
    this.email = email;
    }

    @Override
    public String toString() {
    return customerId + " | " + name + " | " + email;
```

```
    }
}

class Repository<T> {
    private final HashMap<String, T> store = new HashMap<>();

    private String getId(T entity) {
    for (Field field : entity.getClass().getDeclaredFields()) {     if
    (field.isAnnotationPresent(Id.class)) {
                    try {
                        field.setAccessible(true);
                        return String.valueOf(field.get(entity));
                    } catch (IllegalAccessException e) {
                        throw new RuntimeException(e);
                    }
        }
    }
    throw new IllegalArgumentException(
        "No @Id field in " + entity.getClass().getSimpleName()); }

    public T save(T entity) {
    store.put(getId(entity), entity);
    return entity;
    }

    public T findById(String id) {
    return store.get(id);
    }

    public ArrayList<T> findAll() {
    return new ArrayList<>(store.values()); }

    public boolean delete(String id) {
```

```
        return store.remove(id) != null;
        }

        @SafeVarargs
        public final void saveAll(T... items) {
        for (T item : items) {
            save(item);
        }
        }

        public int count() {
        return store.size();
        }

        public static <N extends Number> double average(N[] values) {
        double sum = 0;
        for (N v : values) {
                sum += v.doubleValue();
        }
        return sum / values.length;
        }
}

public class minibank13 {
        public static void main(String[] args) {
        Repository<Account> accountRepo = new Repository<>();
        accountRepo.saveAll(
                new Account("Krish", 5000),
                new Account("Rahul", 2000),
                new Account("Amit", 9000));

            System.out.println("===== Repository<Account> =====");
            System.out.println("Stored items : " + accountRepo.count());
            System.out.println("findById(\"AC0002\") -> " +
            accountRepo.findById("AC0002"));
```

```
        System.out.println("findById(\"AC0099\") -> " +
        accountRepo.findById("AC0099"));

        System.out.println("\nfindAll() :");
        for (Account a : accountRepo.findAll()) {
            System.out.println(a);
        }

        System.out.println("\ndelete(\"AC0001\") -> " +
        accountRepo.delete("AC0001"));
        System.out.println("Stored items after delete : " +
        accountRepo.count());

        System.out.println("\n===== Repository<Customer> (same class, no
        change) =====");
        Repository<Customer> customerRepo = new Repository<>();
        customerRepo.save(new Customer("Krish", "krish@gmail.com"));
        customerRepo.save(new Customer("Rahul", "rahul@gmail.com"));
        System.out.println("findById(\"CUST102\") -> " +
        customerRepo.findById("CUST102"));
        for (Customer c : customerRepo.findAll()) {
            System.out.println(c);
        }

    System.out.println("\n===== Bounded generic method =====");
    Long[] balances = {5000L, 2000L, 9000L};
    System.out.println("Average balance : " +
    Repository.average(balances));
    }
}
```

Output/SS:

```
===== Repository<Account> =====
Stored items : 3
findById("AC0002") -> AC0002 | Rahul | 2000
findById("AC0099") -> null

findAll() :
AC0001 | Krish | 5000
AC0003 | Amit | 9000
AC0002 | Rahul | 2000

delete("AC0001") -> true
Stored items after delete : 2

===== Repository<Customer> (same class, no change) =====
findById("CUST102") -> CUST102 | Rahul | rahul@gmail.com
CUST102 | Rahul | rahul@gmail.com
CUST101 | Krish | krish@gmail.com

===== Bounded generic method =====
Average balance : 5333.333333333333
```

Key Questions and Solution

1.What problem do generics solve compared with using Object?

Answer: Generics give compile-time type safety and need no casting, unlike Object.

2.What is a bounded type parameter, and why is <N extends Number> useful?

Answer: It limits the type to Number and its subclasses, so doubleValue() can be used safely.

3.What is type erasure?

Answer: The compiler removes generic type information after checking, so at run time Box<Integer> and Box<String> are both Box.

PRACTICAL – 14

Problem Statement:

1. Assemble and test: build a tiny layered app (a model, a service using an ArrayList, and a menu main), then write two or three JUnit @Test methods checking that adding increases the count and an invalid action throws — practice before testing MiniBank.

Code (prac14_1.java):

```
import java.util.ArrayList;
import java.util.Scanner;

class Task {
    private final String title;
    private boolean done;

    Task(String title) {
    this.title = title;
    }

    String getTitle() { return title; }
    boolean isDone() { return done; }
    void markDone()  { done = true; }

    public String toString() {
    return (done ? "[x] " : "[ ] ") + title;
    }
}

class TaskService {
    private final ArrayList<Task> tasks = new ArrayList<>();

    void add(String title) {
    if (title == null || title.isBlank()) {
        throw new IllegalArgumentException("Title cannot be empty"); }
    tasks.add(new Task(title));
```

```
    }

    void markDone(int index) {
    if (index < 1 || index > tasks.size()) {
        throw new IllegalArgumentException("No task number " + index); }
    tasks.get(index - 1).markDone();
    }

    int count() {
    return tasks.size();
    }

    ArrayList<Task> getAll() {
    return tasks;
    }
}

public class prac14_1 {
    public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
    TaskService service = new TaskService();
    boolean running = true;

        while (running) {
            System.out.println("\n== TASK MENU ==");
            System.out.println("1. Add task");
            System.out.println("2. Mark task done");
            System.out.println("3. List tasks");
            System.out.println("4. Exit");
            System.out.print("Choice : ");
            String choice = sc.nextLine().trim();

            try {
                switch (choice) {
```

```
                    case "1" -> {
                        System.out.print("Title : ");
                        service.add(sc.nextLine());
                        System.out.println("Task added. Total = " +
                        service.count());
                    }
                    case "2" -> {
                        System.out.print("Task number : ");
                        service.markDone(Integer.parseInt(sc.nextLine().trim(
                        )));
                        System.out.println("Task marked done.");
                    }
                    case "3" -> {
                        int i = 1;
                        for (Task t : service.getAll()) {
                            System.out.println(i++ + ". " + t);
                        }
                    }
                    case "4" -> {
                        running = false;
                        System.out.println("Goodbye.");
                    }
                    default -> System.out.println("Invalid choice.");
                }
            } catch (IllegalArgumentException e) {
                System.out.println("Error : " + e.getMessage());
            }
        }
    }
}
```

Test code (TaskServiceTest.java):

```
import static org.junit.Assert.*;
import org.junit.Test;

public class TaskServiceTest {
```

```
    @Test
    public void addingATaskIncreasesTheCount() {
    TaskService service = new TaskService();
    service.add("Buy milk");
    service.add("Submit lab file");
    assertEquals(2, service.count());
    }

    @Test
    public void emptyTitleThrowsException() {
    TaskService service = new TaskService();
    assertThrows(IllegalArgumentException.class, () -> service.add(" ")); }

    @Test
    public void invalidTaskNumberThrowsException() {
    TaskService service = new TaskService();
    service.add("Only task");
    assertThrows(IllegalArgumentException.class, () ->
    service.markDone(5));
    }

    @Test
    public void markDoneChangesTheTaskState() {
    TaskService service = new TaskService();
    service.add("Revise Java");
    service.markDone(1);
    assertTrue(service.getAll().get(0).isDone());    }
}
```

Output/SS:

```
== TASK MENU ==
1. Add task
2. Mark task done
3. List tasks
4. Exit
Choice : 1
Title : Buy milk
Task added. Total = 1

== TASK MENU ==
1. Add task
2. Mark task done
3. List tasks
4. Exit
Choice : 1
Title : Submit lab file
Task added. Total = 2

== TASK MENU ==
1. Add task
2. Mark task done
3. List tasks
4. Exit
Choice : 1
Title :
Error : Title cannot be empty

== TASK MENU ==
1. Add task
2. Mark task done
3. List tasks
4. Exit
Choice : 3
```

```
1. [ ] Buy milk
2. [ ] Submit lab file

== TASK MENU ==
1. Add task
2. Mark task done
3. List tasks
4. Exit
Choice : 2
Task number : 1
Task marked done.

== TASK MENU ==
1. Add task
2. Mark task done
3. List tasks
4. Exit
Choice : 2
Task number : 9
Error : No task number 9

== TASK MENU ==
1. Add task
2. Mark task done
3. List tasks
4. Exit
Choice : 3
1. [x] Buy milk
2. [ ] Submit lab file

== TASK MENU ==
1. Add task
2. Mark task done
3. List tasks
```

```
4. Exit
Choice : 7
Invalid choice.

== TASK MENU ==
1. Add task
2. Mark task done
3. List tasks
4. Exit
Choice : 4
Goodbye.
```

```
JUnit version 4.13.2
....
Time: 0.02

OK (4 tests)
```

MINIBANK-14

Problem Statement:

All the parts now exist. In this final capstone lab you assemble MiniBank and finish it to a deliverable standard: a BankService wires the repository, validation and exceptions together and the Practical 1 menu calls real operations; you add safe batch processing and the report; and you test and package the application. This is the Milestone 3 final checkpoint and viva.

1.Create a class named BankService that uses Repository<Account>, the Validator and the exception classes to provide openAccount, deposit, withdraw, transfer, statement and listAccounts.

2.Connect the Practical 1 menu so each option calls the matching BankService method; load saved accounts on start-up (Practical 11) and save them on exit.

3.Use the TransactionProcessor (Practical 10) over the thread-safe store (Practical 12) to process a batch of transactions safely, and add the report (Practical 11) as a menu option.

4.Write a few JUnit tests using @Test: a deposit increases the balance; a withdrawal over the limit throws an exception; many concurrent deposits keep the balance correct.

5.(Optional) Add an @Audit annotation and a reflective logger that records which service method was called.

6.Build the final runnable JAR and complete the README.

Code (minibank14.java):

```
import java.io.*;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;
import java.lang.reflect.Field;
import java.nio.charset.StandardCharsets;
import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.Scanner;
import java.util.concurrent.ConcurrentHashMap;
```

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;
import java.util.regex.Pattern;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Id {
}

class BankException extends Exception {
    BankException(String message) {
    super(message);
    }
}

class InsufficientFundsException extends BankException {
    private final long shortfall;

    InsufficientFundsException(long shortfall) {
    super("Insufficient funds, short by " + shortfall);
    this.shortfall = shortfall;
    }

    long getShortfall() {
    return shortfall;
    }
}

class AccountNotFoundException extends BankException {
    AccountNotFoundException(String accountNumber) {
    super("Account not found : " + accountNumber);      }
}

class InvalidAmountException extends BankException {
```

```
    InvalidAmountException(String message) {
    super(message);
    }
}

class Account implements Serializable {
    private static final long serialVersionUID = 1L;
    private static int counter = 1;

    @Id
    private final String accountNumber;
    private final String ownerName;
    private long balance;

    Account(String ownerName, long balance) {
    this.accountNumber = String.format("AC%04d", counter++);
    this.ownerName = ownerName;
    this.balance = balance;
    }

    static void syncCounter(int next) {
    if (next > counter) {
        counter = next;
    }
    }

    public String getAccountNumber() { return accountNumber; }
    public String getOwnerName()     { return ownerName; }
    public synchronized long getBalance() { return balance; }

    public synchronized void deposit(long amount) throws
    InvalidAmountException {
    if (amount <= 0) {
        throw new InvalidAmountException("Amount must be greater than
        zero");
    }
```

```
        balance += amount;
        }

        public synchronized void withdraw(long amount)
            throws InsufficientFundsException, InvalidAmountException { if
        (amount <= 0) {
            throw new InvalidAmountException("Amount must be greater than
            zero");
        }
        if (amount > balance) {
            throw new InsufficientFundsException(amount - balance);
        }
        balance -= amount;
        }

        @Override
        public String toString() {
        return accountNumber + " | " + ownerName + " | " + getBalance();
        }
}

class Validator {
        private static final Pattern MOBILE = Pattern.compile("^[6-9][0-9]{9}$");
        private static final Pattern EMAIL =
        Pattern.compile("^[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+\\.[A-Za-
        z]{2,}$");

        public static boolean isValidMobile(String mobile) {
        return mobile != null && MOBILE.matcher(mobile).matches(); }

        public static boolean isValidEmail(String email) {
        return email != null && EMAIL.matcher(email).matches();
        }
}
```

```
enum TransactionType {
    DEPOSIT, WITHDRAW, TRANSFER
}

record Command(TransactionType type, String accountNumber, long amount) {
}

class CommandParser {
    public static Command parse(String line) {
    String[] parts = line.trim().split("\\s+");
    if (parts.length != 3) {
            throw new IllegalArgumentException("Expected: TYPE
        ACCOUNT AMOUNT");
    }
    return new Command(TransactionType.valueOf(parts[0]),
        parts[1], Long.parseLong(parts[2]));        }
}

class Repository<T> {
    private final ConcurrentHashMap<String, T> store = new
    ConcurrentHashMap<>();

    private String getId(T entity) {
    for (Field field : entity.getClass().getDeclaredFields()) {     if
    (field.isAnnotationPresent(Id.class)) {
                try {
                    field.setAccessible(true);
                    return String.valueOf(field.get(entity));
                } catch (IllegalAccessException e) {
                    throw new RuntimeException(e);
                }
        }
    }
    throw new IllegalArgumentException("No @Id field found");
```

```
    }

    public T save(T entity)       { store.put(getId(entity), entity); return entity; }
    public T findById(String id) { return store.get(id); }
    public ArrayList<T> findAll() { return new ArrayList<>(store.values()); }
    public boolean delete(String id) { return store.remove(id) != null; }
}

class StatePersister {
    public static void save(Account[] accounts, Path file) throws IOException {
    try (ObjectOutputStream out =
                    new ObjectOutputStream(Files.newOutputStream(file))) {
            out.writeObject(accounts);
    }
    }

    public static Account[] load(Path file)
            throws IOException, ClassNotFoundException {
    try (ObjectInputStream in =
                    new ObjectInputStream(Files.newInputStream(file))) {
            return (Account[]) in.readObject();
    }
    }
}

class TransactionLog {
    public static synchronized void append(Path file, String line) throws
    IOException {
    Files.write(file,
            (line +
            System.lineSeparator()).getBytes(StandardCharsets.UTF_8),
        StandardOpenOption.CREATE, StandardOpenOption.APPEND);    }
}

class ReportGenerator {
```

```
public static String generate(Path logsDir, Path reportFile) throws
IOException {
long totalDeposits = 0;
long totalWithdrawals = 0;
StringBuilder report = new StringBuilder();
report.append("===== END OF DAY REPORT =====\n");

    ArrayList<Path> files = new ArrayList<>();
    try (DirectoryStream<Path> stream =
    Files.newDirectoryStream(logsDir)) {
        for (Path p : stream) {
            files.add(p);
        }
    }
    Collections.sort(files);

    for (Path file : files) {
        BasicFileAttributes attr =
            Files.readAttributes(file, BasicFileAttributes.class);   if
    (!file.toString().endsWith(".log") || attr.size() == 0) {
        continue;
        }
        report.append("File : ").append(file.getFileName())
                .append(" (").append(attr.size()).append(" bytes)\n");

        try (BufferedReader br = Files.newBufferedReader(file)) {
            String line;
            while ((line = br.readLine()) != null) {
                String[] parts = line.trim().split("\\s+");
                if (parts.length != 3) {
                    continue;
                }
                if (parts[0].equals("DEPOSIT")) {
                    totalDeposits += Long.parseLong(parts[2]);
            } else if (parts[0].equals("WITHDRAW")) {
```

```
                        totalWithdrawals += Long.parseLong(parts[2]);
                    }
                }
            }
        }
        report.append("Total deposits  :
        ").append(totalDeposits).append("\n");
        report.append("Total withdrawals :
        ").append(totalWithdrawals).append("\n");
        report.append("Net change       : ")
                .append(totalDeposits - totalWithdrawals).append("\n");

    Files.writeString(reportFile, report.toString());
    return report.toString();
    }
}

class TransactionProcessor {
    private final ExecutorService pool = Executors.newFixedThreadPool(4);

    public void submit(Runnable task) {
    pool.execute(task);
    }

    public void stop() throws InterruptedException {
    pool.shutdown();
    pool.awaitTermination(1, TimeUnit.MINUTES);
    }
}

class BankService {
    private final Repository<Account> repo = new Repository<>();
    private final Path logFile;

    BankService(Path logFile) {
    this.logFile = logFile;
```

```
}

private Account find(String accountNumber) throws
AccountNotFoundException {
Account a = repo.findById(accountNumber);
if (a == null) {
    throw new AccountNotFoundException(accountNumber); }
return a;
}

private void log(String line) {
try {
    TransactionLog.append(logFile, line);
} catch (IOException e) {
    System.out.println("Warning : could not write log (" +
    e.getMessage() + ")");
}
}

public Account openAccount(String name, String mobile, String email,
long opening)
    throws BankException {
if (name == null || name.isBlank()) {
    throw new BankException("Name cannot be empty");
}
if (!Validator.isValidMobile(mobile)) {
    throw new BankException("Invalid mobile number");
}
if (!Validator.isValidEmail(email)) {
    throw new BankException("Invalid email address");
}
if (opening < 0) {
    throw new InvalidAmountException("Opening balance cannot be
    negative");
```

```
}
Account account = new Account(name, opening);
repo.save(account);
return account;
}

public void deposit(String accountNumber, long amount) throws
BankException {
find(accountNumber).deposit(amount);
log("DEPOSIT " + accountNumber + " " + amount);
}

public void withdraw(String accountNumber, long amount) throws
BankException {
find(accountNumber).withdraw(amount);
log("WITHDRAW " + accountNumber + " " + amount); }

public void transfer(String from, String to, long amount) throws
BankException {
Account a = find(from);
Account b = find(to);
a.withdraw(amount);
try {
    b.deposit(amount);
} catch (BankException e) {
    a.deposit(amount);
    throw e;
}
log("WITHDRAW " + from + " " + amount);
log("DEPOSIT " + to + " " + amount);
}

public String statement(String accountNumber) throws BankException {
Account a = find(accountNumber);
```

```
        StringBuilder sb = new StringBuilder();
        sb.append("----- Account Statement ----- \n");
        sb.append("Account Number :
        ").append(a.getAccountNumber()).append("\n");
        sb.append("Holder Name :
        ").append(a.getOwnerName()).append("\n");
        sb.append("Balance       : ").append(a.getBalance()).append("\n");
        sb.append("                ");
    return sb.toString();
    }

    public ArrayList<Account> listAccounts() {
    ArrayList<Account> list = repo.findAll();
    list.sort(Comparator.comparing(Account::getAccountNumber));
    return list;
    }

    public ArrayList<Account> searchByOwner(String text) {
    ArrayList<Account> result = new ArrayList<>();
    for (Account a : repo.findAll()) {
        if
        (a.getOwnerName().toLowerCase().contains(text.toLowerCase()))   {
                result.add(a);
        }
    }
    return result;
    }

    public void saveState(Path file) throws IOException {
    StatePersister.save(repo.findAll().toArray(new Account[0]), file); }

    public int loadState(Path file) throws IOException,
    ClassNotFoundException {
    Account[] loaded = StatePersister.load(file);
```

```
        int max = 0;
        for (Account a : loaded) {
            repo.save(a);
            max = Math.max(max,
                    Integer.parseInt(a.getAccountNumber().substring(2)));
        }
        Account.syncCounter(max + 1);
        return loaded.length;
        }

        public void processBatch(ArrayList<String> lines) throws
        InterruptedException {
        TransactionProcessor processor = new TransactionProcessor();
        for (String line : lines) {
                final Command cmd = CommandParser.parse(line);
                processor.submit(() -> {
                    try {
                        if (cmd.type() == TransactionType.DEPOSIT) {
                            deposit(cmd.accountNumber(), cmd.amount());
                    } else if (cmd.type() == TransactionType.WITHDRAW) {
                        withdraw(cmd.accountNumber(), cmd.amount());
                    }
                    } catch (BankException e) {
                        System.out.println("Batch error : " + e.getMessage());
                }
                });
        }
        processor.stop();
        }
}

public class minibank14 {
        static Scanner sc = new Scanner(System.in);

        static String ask(String prompt) {
```

```
System.out.print(prompt);
return sc.nextLine().trim();
}

public static void main(String[] args) throws Exception {
    Path dataDir = Paths.get("data14");
    Path logsDir = dataDir.resolve("logs");
    Path accountsFile = dataDir.resolve("accounts.dat");
    Path reportFile = dataDir.resolve("report.txt");
    Files.createDirectories(logsDir);

    BankService service = new
    BankService(logsDir.resolve("transactions.log"));

    System.out.println("BankInfo[name=MINIBANK,
    branch=ANAND]");
    if (Files.exists(accountsFile)) {
        int n = service.loadState(accountsFile);
        System.out.println("Loaded " + n + " saved account(s).");
    }

    boolean running = true;
    while (running) {
        System.out.println("\n==MINIBANK MENU==");
        System.out.println("1. Open Account");
        System.out.println("2. Deposit");
        System.out.println("3. Withdraw");
        System.out.println("4. Transfer");
        System.out.println("5. Statement");
        System.out.println("6. List Accounts");
        System.out.println("7. Batch Process (100 deposits of 10)");
        System.out.println("8. End-of-day Report");
        System.out.println("9. Exit");
        String choice = ask("Enter your choice : ");

        try {
```

```
switch (choice) {
    case "1" -> {
        String name = ask("Name : ");
        String mobile = ask("Mobile : ");
        String email = ask("Email : ");
        long opening = Long.parseLong(ask("Opening
        balance : "));
        Account a = service.openAccount(name, mobile,
    email, opening);
        System.out.println("Account opened : " + a);
    }
    case "2" -> {
        String no = ask("Account number : ");
        service.deposit(no, Long.parseLong(ask("Amount :
        ")));
        System.out.println("Deposit successful.");
    }
    case "3" -> {
        String no = ask("Account number : ");
        service.withdraw(no, Long.parseLong(ask("Amount :
        ")));
        System.out.println("Withdrawal successful.");
    }
    case "4" -> {
        String from = ask("From account : ");
        String to = ask("To account : ");
        service.transfer(from, to,
        Long.parseLong(ask("Amount : ")));
        System.out.println("Transfer successful.");
    }
    case "5" -> System.out.println(
        service.statement(ask("Account number : ")));
    case "6" -> {
        for (Account a : service.listAccounts()) {
            System.out.println(a);
```

```
                    }
                }
                case "7" -> {
                    String no = ask("Account number : ");
                    ArrayList<String> lines = new ArrayList<>();
                for (int i = 0; i < 100; i++) {
                        lines.add("DEPOSIT " + no + " 10");
                    }
                    service.processBatch(lines);
                    System.out.println("Batch finished.");
                    System.out.println(service.statement(no));
                }
                case "8" -> System.out.println(
                    ReportGenerator.generate(logsDir, reportFile));
            case "9" -> {
                    service.saveState(accountsFile);
                    System.out.println("Data saved. THANK YOU FOR
                    USING MINIBANK, GOODBYE");
                    running = false;
                }
                default -> System.out.println("INVALID CHOICE,
            PLEASE ENTER CORRECT CHOICE");
            }
        } catch (InsufficientFundsException e) {
            System.out.println("Withdrawal failed : short by " +
            e.getShortfall());
        } catch (BankException e) {
            System.out.println("Error : " + e.getMessage());
        } catch (NumberFormatException e) {
            System.out.println("Error : please type a valid number");
    }
}
}
}
```

Test code (BankServiceTest.java):

```
import static org.junit.Assert.*;
import org.junit.Test;
import java.nio.file.*;

public class BankServiceTest {

    @Test
    public void depositIncreasesBalance() throws Exception {
    Account a = new Account("Riya", 5000);
    a.deposit(2000);
    assertEquals(7000L, a.getBalance());
    }

    @Test
    public void withdrawOverLimitThrowsException() {
    Account a = new Account("Riya", 1000);
    InsufficientFundsException e = assertThrows(
        InsufficientFundsException.class, () -> a.withdraw(5000));
    assertEquals(4000L, e.getShortfall());
    assertEquals(1000L, a.getBalance());
    }

    @Test
    public void negativeAmountIsRejected() {
    Account a = new Account("Riya", 1000);
    assertThrows(InvalidAmountException.class, () -> a.deposit(-5)); }

    @Test
    public void manyConcurrentDepositsKeepBalanceCorrect() throws
    Exception {
    Account a = new Account("Riya", 0);
    Thread[] threads = new Thread[10];
    for (int i = 0; i < threads.length; i++) {
```

```
        threads[i] = new Thread(() -> {
                try {
                    for (int j = 0; j < 1000; j++) {
                        a.deposit(1);
                    }
                } catch (InvalidAmountException e) {
                    throw new RuntimeException(e);
                }
        });
        threads[i].start();
    }
    for (Thread t : threads) {
        t.join();
    }
    assertEquals(10000L, a.getBalance());
    }

    @Test
    public void invalidMobileIsRejectedByService() {
    BankService service = new BankService(Paths.get("test-log.log"));
    assertThrows(BankException.class,
        () -> service.openAccount("Riya", "12345", "riya@gmail.com", 100));
    }

    @Test
    public void saveAndLoadKeepsDataUnchanged() throws Exception { Path
    file = Files.createTempFile("accounts", ".dat");
    Account a = new Account("Riya", 4000);
    StatePersister.save(new Account[] { a }, file);
    Account[] loaded = StatePersister.load(file);
    assertEquals(1, loaded.length);
    assertEquals(a.getAccountNumber(), loaded[0].getAccountNumber());
    assertEquals(4000L, loaded[0].getBalance());
    Files.delete(file);
```

```
    }
}
```

Output/SS (first run):

```
BankInfo[name=MINIBANK, branch=ANAND]

==MINIBANK MENU==
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Statement
6. List Accounts
7. Batch Process (100 deposits of 10)
8. End-of-day Report
9. Exit
Enter your choice : 1
Name : Krish
Mobile : 9876543210
Email : krish@gmail.com
Opening balance : 5000
Account opened : AC0001 | Krish | 5000

==MINIBANK MENU==
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Statement
6. List Accounts
7. Batch Process (100 deposits of 10)
8. End-of-day Report
9. Exit
Enter your choice : 1
Name : Rahul
Mobile : 9123456780
Email : rahul@gmail.com
Opening balance : 2000
```

```
Account opened : AC0002 | Rahul | 2000

==MINIBANK MENU==
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Statement
6. List Accounts
7. Batch Process (100 deposits of 10)
8. End-of-day Report
9. Exit
Enter your choice : 1
Name : Amit
Mobile : 12345
Email : amit@gmail.com
Opening balance : 3000
Error : Invalid mobile number

==MINIBANK MENU==
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Statement
6. List Accounts
7. Batch Process (100 deposits of 10)
8. End-of-day Report
9. Exit
Enter your choice : 2
Account number : AC0001
Amount : 1500
Deposit successful.
```

```
==MINIBANK MENU==
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Statement
6. List Accounts
7. Batch Process (100 deposits of 10)
8. End-of-day Report
9. Exit
Enter your choice : 3
Account number : AC0002
Amount : 5000
Withdrawal failed : short by 3000

==MINIBANK MENU==
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Statement
6. List Accounts
7. Batch Process (100 deposits of 10)
8. End-of-day Report
9. Exit
Enter your choice : 3
Account number : AC0001
Amount : 1000
Withdrawal successful.

==MINIBANK MENU==
1. Open Account
2. Deposit
3. Withdraw
```

```
4. Transfer
5. Statement
6. List Accounts
7. Batch Process (100 deposits of 10)
8. End-of-day Report
9. Exit
Enter your choice : 4
From account : AC0001
To account : AC0002
Amount : 500
Transfer successful.

==MINIBANK MENU==
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Statement
6. List Accounts
7. Batch Process (100 deposits of 10)
8. End-of-day Report
9. Exit
Enter your choice : 5
Account number : AC0001
----- Account Statement -----
Account Number : AC0001
Holder Name    : Krish
Balance        : 5000
-----------------------------

==MINIBANK MENU==
1. Open Account
2. Deposit
3. Withdraw
```

```
4. Transfer
5. Statement
6. List Accounts
7. Batch Process (100 deposits of 10)
8. End-of-day Report
9. Exit
Enter your choice : 6
AC0001 | Krish | 5000
AC0002 | Rahul | 2500

==MINIBANK MENU==
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Statement
6. List Accounts
7. Batch Process (100 deposits of 10)
8. End-of-day Report
9. Exit
Enter your choice : 7
Account number : AC0002
Batch finished.
----- Account Statement -----
Account Number : AC0002
Holder Name    : Rahul
Balance        : 3500
-----------------------------

==MINIBANK MENU==
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
```

```
5. Statement
6. List Accounts
7. Batch Process (100 deposits of 10)
8. End-of-day Report
9. Exit
Enter your choice : 8
===== END OF DAY REPORT =====
File : transactions.log (1880 bytes)
Total deposits    : 3000
Total withdrawals : 1500
Net change        : 1500


==MINIBANK MENU==
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Statement
6. List Accounts
7. Batch Process (100 deposits of 10)
8. End-of-day Report
9. Exit
Enter your choice : 9
Data saved. THANK YOU FOR USING MINIBANK, GOODBYE
```

Output/SS (second run – saved accounts are loaded on start-up):

```
BankInfo[name=MINIBANK, branch=ANAND]
Loaded 2 saved account(s).

==MINIBANK MENU==
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Statement
6. List Accounts
7. Batch Process (100 deposits of 10)
8. End-of-day Report
9. Exit
Enter your choice : 6
AC0001 | Krish | 5000
AC0002 | Rahul | 3500

==MINIBANK MENU==
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Statement
6. List Accounts
7. Batch Process (100 deposits of 10)
8. End-of-day Report
9. Exit
Enter your choice : 1
Name : Amit
Mobile : 9988776655
Email : amit@gmail.com
Opening balance : 3000
Account opened : AC0003 | Amit | 3000
```

```
==MINIBANK MENU==
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Statement
6. List Accounts
7. Batch Process (100 deposits of 10)
8. End-of-day Report
9. Exit
Enter your choice : 6
AC0001 | Krish | 5000
AC0002 | Rahul | 3500
AC0003 | Amit | 3000

==MINIBANK MENU==
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Statement
6. List Accounts
7. Batch Process (100 deposits of 10)
8. End-of-day Report
9. Exit
Enter your choice : 9
Data saved. THANK YOU FOR USING MINIBANK, GOODBYE
```

JUnit test result:

```
JUnit version 4.13.2
......
Time: 0.068

OK (6 tests)
```

Runnable JAR:

```
BankInfo[name=MINIBANK, branch=ANAND]
Loaded 3 saved account(s).

==MINIBANK MENU==
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Statement
6. List Accounts
7. Batch Process (100 deposits of 10)
8. End-of-day Report
9. Exit
Enter your choice : 6
AC0001 | Krish | 5000
AC0002 | Rahul | 3500
AC0003 | Amit | 3000

==MINIBANK MENU==
1. Open Account
2. Deposit
3. Withdraw
4. Transfer
5. Statement
6. List Accounts
7. Batch Process (100 deposits of 10)
8. End-of-day Report
9. Exit
Enter your choice : 9
Data saved. THANK YOU FOR USING MINIBANK, GOODBYE
```

Key Questions and Solution

1.What is a layered architecture (model, service, user interface), and why is it useful?

Answer: It splits the app into model, service and UI, so each part is easy to change and test.

2.How do you assert in JUnit that a method throws an expected exception? Answer: Use assertThrows(ExceptionClass.class, () -> code); the test passes only if that exception is thrown.

3.What does packaging the application as a runnable JAR provide?

Answer: A JAR bundles all classes into one file that runs with java -jar minibank.jar.



4 Courses

Introduction to Java

Introduction to Object-Oriented Programming with Java

Object-Oriented Hierarchies in Java

Java Class Library

Oct 1, 2026

Krish varsani

has successfully completed the online Specialization

Core Java

In this Core Java specialization, learners gained the knowledge and skills needed to become an object-oriented programmer using the Java language. In the four courses of this specialization, learners mastered the Java programming language and the packages that constitute its rich set of core libraries. Through many hands-on exercises, learners developed their own Java-based applications using beginner to advanced Java functions.

LearnQuest Instructor Network

LearnQuest's Instructor Network is comprised of professionals across hundreds of technology disciplines from around the world.

This certificate attests to the learner's completion of an online program delivered via Coursera. It does not constitute formal enrollment at any university or entity and does not itself grant academic credit, grades, or a degree. Institutions or organizations may, at their discretion, recognize this learning toward their own programs or credentials.

Verify this certificate at: https://coursera.org/verify/specialization/576TJ5OXZ8DL